

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Proyecto 1. Pendrive Multiboot - GNU/Linux - Tools y Windows 10 (32/64 bits)

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 25 de octubre del 2026

Índice

1. Arquitectura del arranque en sistemas computacionales **1**
1.1. Secuencia de inicialización, firmware y particionado 1
1.1.1. Arquitectura BIOS tradicional frente a UEFI 1
1.1.2. Estructuras de almacenamiento masivo: MBR y GPT 3
1.2. Cargadores de arranque y mecanismos de carga del núcleo 4
1.2.1. Arquitectura multietapa en cargadores de arranque 5
1.2.2. Carga del núcleo, discos en memoria y entornos de ejecución previa . 6

2. Tecnologías y herramientas para medios de arranque múltiple **7**
2.1. Arquitectura interna y operación de Ventoy 8
2.1.1. Topología de particionado y aislamiento de funciones 8
2.1.2. Mecanismos de intercepción y enlace dinámico de imágenes 8
2.2. Contraste frente a herramientas alternativas 9
2.2.1. Rufus y la dicotomía entre modos ISO y DD 10
2.2.2. YUMI y el encadenamiento de cargadores propietarios 10
2.2.3. Configuración manual de GNU GRUB con loopback 11

3. Análisis técnico de los entornos integrados **12**
3.1. Entornos de diagnóstico y rescate: Hiren's BootCD 15.2 y Knoppix 8.6 12
3.1.1. Hiren's BootCD 15.2 y la suite de utilerías en modo real 12
3.1.2. Knoppix 8.6: Arquitectura híbrida y eliminación de systemd 13
3.2. Sistemas GNU/Linux para servidor y escritorio (32 y 64 bits) 14
3.2.1. Arquitectura de Linux Server y gestión de servicios 14
3.2.2. Kali Linux: Entorno de auditoría y modo forense 15
3.3. Entorno de instalación de Microsoft Windows 10 (WinPE, 32 y 64 bits) 15
3.3.1. Arquitectura del núcleo híbrido NT 10.0 y el subsistema WinPE 15
3.3.2. Divergencias arquitectónicas entre 32 bits y 64 bits 16

4. Metodología de preparación y despliegue del medio multiboot **18**
4.1. Preparación del dispositivo y despliegue de Ventoy mediante consola 18
4.1.1. Identificación de bloques y análisis de capacidad requerida 18
4.1.2. Instalación de Ventoy con particionado GPT y Secure Boot 19
4.2. Transferencia de imágenes de disco y configuración de parámetros 19
4.2.1. Organización jerárquica y transferencia sincronizada 20
4.2.2. Configuración avanzada mediante directivas ventoy.json 20
4.2.3. Enrolamiento del certificado MOK para Secure Boot 21

5. Protocolo de verificación y pruebas de operatividad **22**
5.1. Entorno de validación y control del menú de arranque 22
5.1.1. Infraestructura de pruebas: Hardware real y emulación con QEMU/KVM 22
5.1.2. Comprobación del menú interactivo de Ventoy 23
5.2. Comprobación de utilerías y sistemas de diagnóstico 23
5.2.1. Validación operativa de Hiren's BootCD 15.2 23
5.2.2. Validación operativa de Knoppix 8.6 24
5.3. Comprobación de sistemas de servidor, escritorio e instalador de Windows . 24
5.3.1. Validación de GNU/Linux Server (Ubuntu Server 22.04 LTS) 24
5.3.2. Validación de Kali Linux: Modo Forense y Persistencia 25

5.3.3. Validación de Microsoft Windows 10 (32 y 64 bits) 25

6. Evaluación comparativa de interoperabilidad y rendimiento 27
6.1. Matriz de compatibilidad entre arquitecturas y modos de firmware 27
6.2. Análisis de sobrecarga de emulación y rendimiento de E/S 28
6.2.1. Sobrecarga de memoria: Mapeo de descriptores frente a disco RAM en bloque 28
6.2.2. Rendimiento de transferencia y latencia en buses USB 2.0 y USB 3.0 29
6.2.3. Preservación de vida útil de la memoria flash frente a métodos tradicionales 30

7. Conclusiones personales 31

Referencias 33

1. Arquitectura del arranque en sistemas computacionales

El proceso de arranque de una computadora constituye una secuencia crítica de transición en la cual el hardware pasa de un estado inerte a un entorno operativo controlado por el núcleo del sistema operativo. Debido a que la memoria principal de acceso aleatorio (RAM) es intrínsecamente volátil y carece de instrucciones válidas al energizar la máquina, la ejecución debe orquestarse a través de programas permanentes almacenados en memorias de sólo lectura o memorias de estado sólido no volátiles, estableciendo un enlace coordinado entre los componentes electrónicos de la placa base, las estructuras de almacenamiento secundario y los gestores de arranque intermedios (Carretero Pérez et al., 2001, §2.2, p. 39). Comprender esta arquitectura es esencial para diseñar y estructurar medios de arranque múltiple capaces de coexistir con diversos estándares de *firmware* y tablas de particionado.

1.1. Secuencia de inicialización, firmware y particionado

Cuando una computadora recibe suministro eléctrico o procesa una señal de reinicio, el hardware genera una señal eléctrica que inicializa los registros internos de la unidad central de procesamiento (CPU) a valores prefijados de fábrica. En la arquitectura x86, el registro de segmento de código (CS) y el puntero de instrucciones (EIP) se configuran para apuntar al denominado vector de reinicio (*reset vector*), ubicado en la dirección física hexadecimal `0xFFFFFFF0`, en el extremo superior de la memoria direccionable de 32 bits, o en `0xFFFF0` en modo real histórico (Deitel et al., 2004, §2.4.3, p. 76). En dicha posición reside una instrucción de salto incondicional hacia el inicio del programa de arranque permanente grabado en el circuito integrado de memoria no volátil (ROM o memoria flash) de la placa base.

Este programa permanente ejecuta de manera inmediata la comprobación automática de encendido (POST, *power-on self-test*). Durante la fase POST, el firmware realiza un diagnóstico exhaustivo de la plataforma: verifica la integridad de los registros de la CPU, inicializa los controladores del bus del sistema, calibra y verifica los módulos de memoria RAM disponible, reconoce los dispositivos periféricos indispensables e inicializa los controladores de vídeo y almacenamiento masivo (Carretero Pérez et al., 2001, §2.2, p. 40). Superada la comprobación física, el flujo de control se ramifica según la arquitectura del firmware presente en la placa: el sistema básico de entrada/salida tradicional (BIOS) o la interfaz de firmware extensible unificada (UEFI).

1.1.1. Arquitectura BIOS tradicional frente a UEFI

La BIOS tradicional (frecuentemente denominada BIOS heredada o *Legacy* BIOS) opera confinada al modo real de 16 bits de los procesadores x86, lo cual limita el espacio de direccionamiento accesible a únicamente un megabyte de memoria física y restringe el uso de los registros extendidos de la CPU. Para interactuar con los periféricos durante la fase de inicio, la BIOS establece la tabla de vectores de interrupción (IVT, *interrupt vector table*) en los primeros 1024 bytes de memoria (`0x0000:0x0000` a `0x0000:0x03FF`) y expone rutinas básicas de entrada/salida mediante interrupciones de software, tales como la INT 10h para

funciones de pantalla en modo texto y la INT 13h para operaciones de lectura de bloques de almacenamiento (Carretero Pérez et al., 2001, §2.2, p. 40).

El mecanismo de direccionamiento de disco en la INT 13h tradicional empleaba las coordenadas físicas de cilindro-cabeza-sector (CHS, *cylinder-head-sector*), con un límite rígido de 1024 cilindros, 256 cabezas y 63 sectores, imponiendo una barrera máxima de capacidad de almacenamiento de aproximadamente 8.4 gigabytes. Aunque la introducción de las extensiones INT 13h con direccionamiento por bloque lógico (LBA, *logical block addressing*) mitigó esta restricción permitiendo descriptores de bloque de 32 y 48 bits, la BIOS mantuvo deficiencias estructurales insalvables: lentitud en la inicialización de buses modernos, incapacidad para ejecutar controladores complejos de red o almacenamiento en la fase previa al sistema operativo y ausencia absoluta de protocolos de verificación criptográfica para validar la autenticidad del código de arranque.

Para superar las limitaciones físicas de la BIOS, el consorcio de la industria formuló el estándar UEFI (Unified Extensible Firmware Interface Forum, 2024). A diferencia de la BIOS, UEFI se ejecuta desde su primera instrucción en modo protegido de 32 bits (IA-32) o en modo largo de 64 bits (x86_64 o AArch64), eliminando por completo la barrera del megabyte y permitiendo acceder a la totalidad de la memoria RAM del sistema antes de cargar el sistema operativo. La arquitectura de UEFI es completamente modular y orientada a objetos: el firmware provee una capa de abstracción de hardware a través de controladores de dispositivos escritos en código nativo o en el lenguaje intermedio EBC (*EFI Byte Code*), lo que permite montar sistemas de archivos, gestionar interfaces de red completas (TCP/IP, DHCP, PXE) y renderizar gráficos de alta resolución mediante el protocolo de salida gráfica (GOP, *graphics output protocol*).

Las funciones expuestas por la interfaz UEFI se dividen formalmente en dos clases de servicios claramente delimitadas:

1. **Servicios de arranque (*Boot Services*):** Incluyen funciones de asignación de memoria dinámica, gestión de eventos y temporizadores, protocolos de consola y controladores de sistemas de archivos (con soporte nativo para FAT12, FAT16 y FAT32). Estos servicios residen en memoria mientras el cargador localiza y prepara el núcleo; en cuanto el sistema operativo toma el control pleno del hardware, el cargador invoca la función de firmware `ExitBootServices()`, liberando la memoria ocupada por dichos servicios para uso del sistema operativo (Unified Extensible Firmware Interface Forum, 2024, §13, p. 38).
2. **Servicios en tiempo de ejecución (*Runtime Services*):** Comprenden rutinas de soporte que permanecen residentes en memoria virtual incluso tras el inicio del sistema operativo. Proporcionan acceso a variables no volátiles almacenadas en la NVRAM (utilizadas por herramientas del sistema operativo como `efibootmgr` para consultar y modificar el orden de arranque), consulta y actualización del reloj de tiempo real (RTC) del sistema y control de estados de energía y reinicio mediante las tablas ACPI (*advanced configuration and power interface*).

Asimismo, UEFI incorpora la arquitectura de arranque seguro (*Secure Boot*), un mecanismo de validación criptográfica concebido para impedir la ejecución de cargadores de arranque, controladores o hipervisores no autorizados o alterados por *malware* en el sector de inicio. En una plataforma con Secure Boot habilitado, el firmware verifica la firma digital en formato PKCS #7 y el hash criptográfico SHA-256 de cada archivo ejecutable en formato PE/COFF (`.efi`) antes de permitir su ejecución. Esta cadena de confianza se apoya en una jerarquía de certificados almacenados en la memoria NVRAM: la clave de plataforma (PK, *platform key*), la clave de intercambio de claves (KEK, *key exchange key*), la base de datos

de firmas autorizadas (`db`) y la base de datos de firmas revocadas (`dbx`). La Tabla 1 sintetiza las diferencias de arquitectura y diseño entre ambos modelos de firmware.

Tabla 1: Comparativa técnica de arquitectura: firmware BIOS tradicional frente a UEFI.

Parámetro	BIOS tradicional (*Legacy*)	Estándar UEFI
Modo de ejecución del procesador	Modo real de 16 bits; limitado estrictamente a 1 megabyte de direccionamiento inicial.	Modo protegido de 32 bits o modo largo de 64 bits nativo desde la primera etapa de ejecución.
Interfaz de entrada/salida	Interrupciones por software del procesador (IVT: INT 10h para vídeo, INT 13h para disco).	Arquitectura orientada a protocolos y controladores modulares (EBC, GOP, pila de red UEFI).
Esquema de particionado nativo	Registro de arranque maestro (MBR); límite estructural de 2.19 terabytes por unidad.	Tabla de particiones GUID (GPT); direccionamiento por bloques de 64 bits hasta 9.4 zettabytes.
Soporte de sistemas de archivos	Carente de abstracción de archivos; lee sectores brutos fijos sin comprender estructuras lógicas.	Controlador de sistema de archivos integrado con soporte nativo de lectura y escritura en volúmenes FAT.
Seguridad e integridad en arranque	Nula; ejecución ciega de cualquier código presente en los primeros 446 bytes del disco.	Validación criptográfica obligatoria mediante firmas digitales X.509 y bases de datos NVRAM (Secure Boot).

1.1.2. Estructuras de almacenamiento masivo: MBR y GPT

La forma en que el firmware localiza e interpreta los datos del sistema operativo depende de la tabla de particiones grabada en el medio de almacenamiento masivo. Históricamente, las arquitecturas basadas en BIOS operan sobre el registro de arranque maestro (MBR, *master boot record*), un sector de 512 bytes localizado en la dirección física de bloque LBA 0 del disco (Silberschatz et al., 2006, §12.5.2, p. 419). El sector MBR se divide estrictamente en tres regiones:

1. **Código de arranque primario (446 bytes):** Instrucciones de código máquina correspondientes al cargador de primera etapa (*bootstrap loader*).
2. **Tabla de particiones primaria (64 bytes):** Cuatro entradas descriptivas de 16 bytes cada una. Cada descriptor almacena el estado de la partición (un byte booteable donde `0x80` indica activa y `0x00` inactiva), coordenadas iniciales CHS (tres bytes), identificador del tipo de partición (un byte, por ejemplo `0x07` para NTFS/exFAT o `0x83` para sistemas Linux nativos), coordenadas finales CHS (tres bytes), dirección LBA inicial del sector (cuatro bytes en formato *little-endian*) y el número total de sectores asignados a dicha partición (cuatro bytes).
3. **Firma de validación (dos bytes):** Palabra mágica hexadecimal fija `0x55AA` en los desplazamientos `0x1FE` y `0x1FF`, requerida para que el firmware considere el sector como válido.

La restricción a cuatro campos de 32 bits para definir el inicio y tamaño en sectores impone el límite intrínseco del MBR: con sectores lógicos convencionales de 512 bytes, la capa-

cidad direccionable máxima equivale a $2^{32} \times 512$ bytes $\approx 2{,}199{,}023{,}255{,}552$ bytes ≈ 2.199 TB. Cualquier capacidad excedente en un disco de mayor tamaño queda completamente inaccesible bajo direccionamiento MBR de 32 bits. Para mitigar el límite de cuatro particiones primarias, el esquema MBR introdujo la partición extendida (identificada comúnmente con el código `0x05` o `0x0F`), dentro de la cual se concatenan registros de arranque extendidos (EBR, *extended boot record*) mediante una lista enlazada de sectores para alojar particiones lógicas secundarias.

En contraposición, el estándar UEFI define la tabla de particiones GUID (GPT, *GUID partition table*), eliminando las limitaciones del MBR mediante el uso de direccionamiento de bloques de 64 bits y asignación de identificadores únicos universales (GUID) de 128 bits para cada partición y tipo de volumen (Unified Extensible Firmware Interface Forum, 2024). La disposición de sectores en un disco particionado bajo GPT presenta una estructura redundante y jerárquica:

- **LBA 0 — MBR protector (*Protective MBR*):** Contiene una tabla MBR ficticia con una única partición de tipo `0xEE` que abarca todo el tamaño del disco. Su propósito técnico es prevenir que utilerías de disco antiguas que no reconozcan GPT interpreten el medio como espacio sin formato y destruyan las particiones existentes.
- **LBA 1 — Cabecera GPT primaria:** Almacena la firma ASCII `EFI PART`, versión de la especificación, tamaño de la cabecera (92 bytes), suma de verificación por redundancia cíclica (CRC32) de la propia cabecera (calculada con el campo de CRC puesto temporalmente en cero), dirección LBA actual (1), dirección LBA de la cabecera de respaldo al final del disco, primer y último bloque utilizables para datos, GUID único del disco, LBA inicial de la matriz de particiones (fijado comúnmente en el LBA 2) y el número de entradas disponibles (mínimo 128).
- **LBA 2 al 33 — Matriz de entradas de partición:** Reserva 32 sectores contiguos para alojar un arreglo de al menos 128 entradas de partición de 128 bytes cada una. Cada entrada almacena el GUID de tipo de partición (16 bytes), el GUID único de la partición (16 bytes), el primer y último LBA de 64 bits que delimitan el volumen, una máscara de atributos de 64 bits (de sólo lectura, oculta, no automontable) y el nombre de la partición codificado en formato Unicode UTF-16LE de hasta 36 caracteres.
- **LBA final y sectores previos — Estructura GPT de respaldo:** En los últimos 33 sectores del disco físico se aloja una réplica íntegra de la matriz de particiones y la cabecera GPT secundaria, lo que confiere tolerancia a fallos ante corrupciones de sectores iniciales.

Bajo GPT y direccionamiento de 64 bits, una plataforma puede direccionar hasta 2^{64} sectores, lo cual representa teóricamente 9.44×10^{21} bytes (8 zebibytes), superando holgadamente los límites contemporáneos del hardware. Asimismo, toda plataforma UEFI requiere la existencia de la partición del sistema EFI (ESP, *EFI system partition*), formateada obligatoriamente en un sistema de archivos compatible con FAT (habitualmente FAT32), identificada mediante el GUID formal `C12A7328-F81F-11D2-BA4B-00A0C93EC93B`. En la ESP residen los cargadores de arranque en formato ejecutable PE32+ (`.efi`), configurados en rutas estandarizadas de contingencia como `\EFI\BOOT\BOOTX64.EFI` para arquitecturas de 64 bits o `\EFI\BOOT\BOOTIA32.EFI` para sistemas de 32 bits.

1.2. Cargadores de arranque y mecanismos de carga del núcleo

El firmware de la computadora carece de las estructuras necesarias para cargar directamente el núcleo monolítico o híbrido de un sistema operativo moderno. En el modelo clásico

de PC, la BIOS desconoce la semántica de los sistemas de archivos complejos (como ext4, XFS, Btrfs o NTFS), por lo que su función se reduce a transferir la ejecución al sector cero del disco seleccionado en el orden de arranque (Silberschatz et al., 2006, §2.10, p. 64). Esta separación modular demanda la intervención de programas intermedios especializados denominados cargadores de arranque (*bootloaders*), los cuales operan de manera escalonada para abstraer el almacenamiento y materializar el núcleo en la memoria RAM.

1.2.1. Arquitectura multietapa en cargadores de arranque

En plataformas basadas en BIOS y MBR, el cargador se enfrenta a la severa restricción de espacio impuesta por los 446 bytes disponibles en el sector LBA 0. En dicha extensión de memoria es imposible albergar rutinas de interfaz de usuario, tablas de descompresión o controladores de sistemas de archivos. Por este motivo, herramientas avanzadas como GNU GRUB (*GRand Unified Bootloader*) implementan una carga multietapa estructurada (Free Software Foundation, 2024a):

- **Etapa 1 (*Stage 1*):** Consiste en el código máquina ensamblado en el archivo `boot.img`, escrito estrictamente dentro de los primeros 446 bytes del MBR. Su función exclusiva consiste en leer de manera lineal los sectores del disco que contienen la siguiente etapa (utilizando llamadas directas a la interrupción BIOS INT 13h) y bifurcar hacia su dirección de carga.
- **Etapa 1.5 (*Stage 1.5* o `core.img`):** Se ubica físicamente en el espacio no particionado comprendido entre el MBR (LBA 0) y el sector inicial de la primera partición formateada (típicamente alineada en el LBA 2048, lo que deja un espacio libre o *gap* de 2047 sectores contiguos, equivalente a 1 megabyte). La imagen `core.img` incluye el núcleo operativo básico de GRUB, controladores de lectura cruda y el módulo del sistema de archivos específico donde se encuentra instalado el directorio `/boot/`.
- **Etapa 2 (*Stage 2*):** Una vez que `core.img` es capaz de interpretar la estructura lógica de directorios, monta el volumen de arranque, localiza el directorio `/boot/grub/`, enlaza dinámicamente sus módulos en formato binario ELF (`.mod`), despliega la interfaz gráfica o de consola en pantalla e interpreta el archivo de comandos `grub.cfg`.

En contraparte, el arranque bajo UEFI prescinde de las etapas intermedias 1 y 1.5. Debido a que el firmware UEFI incorpora internamente controladores para leer volúmenes con formato FAT32, localiza y lee de forma directa el archivo ejecutable `grubx64.efi` desde la partición ESP, asignándole memoria en modo largo y cediéndole la ejecución de inmediato sin depender de sectores ocultos en el disco.

Para independizar el proceso de inicio de las particularidades de cada cargador propietario, la Free Software Foundation formuló la especificación *Multiboot* (Free Software Foundation, 2024b). Este estándar define un protocolo unificado de comunicación entre el cargador y el kernel. El archivo binario del núcleo debe incorporar en sus primeros 8192 bytes una cabecera de metadatos (*Multiboot header*) con una palabra mágica hexadecimal (`0x1BADB002` para Multiboot 1 o `0xE85250D6` para Multiboot 2). Al momento de transferir el control, el procesador se encuentra en modo protegido de 32 bits con la paginación deshabilitada, el registro EAX cargado con el número mágico de confirmación y el registro EBX apuntando a una estructura en memoria física que entrega al núcleo:

- El mapa formal de memoria RAM física del sistema (obtenido mediante llamadas a la función E820 de BIOS o `GetMemoryMap()` de UEFI), distinguiendo zonas de memoria utilizables de regiones reservadas por el firmware.
- Los argumentos y directivas de la línea de comandos pasados al núcleo.

- La dirección base y longitud de los módulos auxiliares cargados en memoria RAM, entre los cuales destaca la imagen del disco RAM inicial.
- Parámetros de configuración del búfer de vídeo (resolución espacial, profundidad cromática y dirección física de la memoria de fotogramas).

1.2.2. Carga del núcleo, discos en memoria y entornos de ejecución previa

El núcleo del sistema operativo GNU/Linux se distribuye habitualmente como una imagen binaria comprimida denominada `vmlinuz`. Esta imagen integra una cabecera de inicialización y descompresión ligada a la carga útil del núcleo compilado. Sin embargo, para que el núcleo pueda montar su sistema de archivos raíz definitivo (almacenado comúnmente sobre matrices RAID, particiones cifradas con LUKS, volúmenes lógicos LVM o sistemas de archivos de red), requiere controladores específicos para el hardware de almacenamiento y el bus PCI Express. Compilar todos los controladores posibles directamente dentro del núcleo estático generaría una imagen inviable por consumo de memoria.

La solución arquitectónica consiste en delegar esta fase al espacio de usuario temprano (*early userspace*) mediante un disco en memoria virtual (Landley & The Linux Kernel Organization, 2024). Históricamente se empleaba el esquema `initrd` (*initial ramdisk*), que consistía en una imagen de disco con formato ext2 simulada sobre un dispositivo de bloques en memoria (`/dev/ramX`). Dicho diseño exigía un controlador de disco RAM en el núcleo, asignaba un tamaño fijo de memoria difícilmente redimensionable y recurría a la llamada al sistema `pivot_root` para intercambiar la raíz temporal por el disco permanente.

El modelo contemporáneo en entornos Linux corresponde al sistema de archivos en RAM inicial, denominado `initramfs` (Landley & The Linux Kernel Organization, 2024). El archivo `initramfs` es un archivo de intercambio en formato cpio comprimido (mediante algoritmos gzip, xz o zstd). Al arrancar el sistema:

1. El cargador de arranque deposita el archivo binario del núcleo `vmlinuz` y el archivo `initramfs` en zonas separadas de la memoria física.
2. El núcleo se descomprime a sí mismo, activa el subsistema de paginación y crea en memoria un sistema de archivos raíz primitivo denominado `rootfs` (un sistema de archivos virtual basado en `ramfs`).
3. El kernel extrae el contenido del cpio directamente dentro de `rootfs` y ejecuta el programa `/init` como proceso con identificador de proceso 1 (PID 1).
4. El script `/init` carga los módulos del kernel necesarios para reconocer la controladora de disco, detecta la partición raíz del sistema operativo y la monta en un directorio temporal, típicamente denominado `/newroot`.
5. Debido a que `rootfs` constituye la raíz primaria del núcleo y no puede desmontarse mediante `pivot_root`, el proceso invoca la utilidad especializada `switch_root`. Esta herramienta borra de manera recursiva todos los archivos del `initramfs` presentes en `rootfs` para liberar la memoria RAM, mueve el punto de montaje de `/newroot` sobre la raíz `/`, efectúa un cambio de raíz (*chroot*) y reemplaza el proceso actual con el ejecutable `/sbin/init` o `systemd` mediante la llamada `execve()`.

Para sistemas en vivo (*Live*) que operan desde medios ópticos o memorias USB sin instalación previa, como Knoppix, la compresión de datos es indispensable para alojar entornos gráficos completos dentro de medios de capacidad limitada. Para este fin, Klaus Knopper diseñó el módulo del núcleo `cloop` (*Compressed Loopback Device*) (Knopper, 2024). A diferencia de un sistema de archivos convencional, `cloop` opera a nivel de dispositivo de bloques virtual de sólo lectura (`/dev/cloopX`). Una imagen completa de sistema de archivos (tal co-

mo un volumen ext2 o ISO 9660) se subdivide en bloques contiguos de tamaño uniforme (usualmente 64 o 128 kilobytes), comprimiendo cada bloque de forma individual mediante zlib e integrando una tabla de índices en la cabecera del archivo. Esto permite al controlador del núcleo buscar y descomprimir al vuelo cualquier bloque solicitado con tiempo de acceso aleatorio casi transparente, sin requerir la descompresión total de la imagen en la memoria RAM. Combinado con controladores de unión como AUFS u OverlayFS montados sobre un disco en RAM volátil (`tmpfs`), el sistema operativo proporciona un entorno de lectura y escritura completamente operativo sobre un medio físicamente inmutable.

En el ecosistema de Microsoft Windows, la fase de prearranque y mantenimiento se instrumenta a través del entorno de preinstalación de Windows (WinPE, *Windows Preinstallation Environment*) (Microsoft Corporation, 2024c). Durante el inicio de un instalador o medio de rescate de Windows:

1. El administrador de arranque (`bootmgr` bajo BIOS o `bootmgr.efi` en UEFI) consulta la base de datos de configuración de arranque (BCD, *boot configuration data*) (Microsoft Corporation, 2024a).
2. El gestor localiza la imagen comprimida del sistema `boot.wim` (en formato *Windows Imaging Format*) y la transfiere íntegramente a un segmento de la memoria física.
3. El controlador en modo núcleo de disco en memoria `ramdisk.sys` mapea el espacio de RAM que contiene la imagen WIM, exponiéndola ante el subsistema de almacenamiento como un volumen virtual de disco duro.
4. El cargador del sistema operativo (`winload.efi` o `winload.exe`) inicializa la capa de abstracción de hardware (HAL, *hardware abstraction layer*) y transfiere el control al núcleo ejecutivo de Windows (`ntoskrnl.exe`).
5. El núcleo monta el sistema de archivos del volumen WIM virtualizado en RAM y lanza el subsistema de usuario mediante `winpeshl.exe`, ejecutando scripts de diagnóstico y consolas de administración sin interactuar destructivamente con los discos duros de la máquina anfitriona.

Esta correlación entre cargadores multietapa, discos RAM virtuales y particionado estructurado proporciona el fundamento técnico sobre el cual operan las herramientas modernas de arranque múltiple en medios USB.

2. Tecnologías y herramientas para medios de arranque múltiple

La coexistencia armónica de múltiples sistemas operativos y entornos de rescate dentro de una única unidad de almacenamiento masivo USB exige mecanismos avanzados de abstracción a nivel de bloques y sistemas de archivos. Durante décadas, la creación de medios de instalación estuvo dominada por utilerías orientadas a la extracción física de imágenes ISO o al volcado crudo sector a sector sobre el dispositivo de almacenamiento, lo que restringía el soporte a un único sistema operativo por unidad o requería esquemas rígidos de encadenamiento con soporte fragmentado entre arquitecturas BIOS y UEFI. El surgimiento de soluciones modernas basadas en emulación al vuelo e intercepción de llamadas del firmware ha transformado este paradigma, permitiendo consolidar bibliotecas heterogéneas de diagnóstico y despliegue sin alterar la estructura original de los archivos de imagen.

2.1. Arquitectura interna y operación de Ventoy

Ventoy es una solución de código abierto diseñada para proporcionar arranque múltiple directo a partir de archivos de imagen (`.iso`, `.wim`, `.img`, `.vhd`) almacenados como archivos ordinarios dentro del sistema de archivos de una memoria USB (Ventoy Project, 2024a). A diferencia de los métodos convencionales de preparación de medios de arranque, Ventoy no descomprime los archivos de imagen ni sobreescribe la estructura de particiones del dispositivo con la geometría del disco óptico de origen.

2.1.1. Topología de particionado y aislamiento de funciones

Para compatibilizar los requerimientos de arranque con la versatilidad de almacenamiento convencional, el instalador de Ventoy configura el medio extraíble en dos particiones primarias dedicadas, soportando tanto tablas de particiones MBR como GPT:

1. **Partición 1 (`Ventoy` — Almacenamiento de usuario):** Ocupa casi la totalidad de la capacidad útil del dispositivo. Por defecto se inicializa con el sistema de archivos exFAT (*extensible file allocation table*), el cual ofrece alta interoperabilidad nativa entre entornos Linux, Windows y macOS, además de admitir archivos de imagen individuales que superen el límite histórico de 4 gigabytes de FAT32. No obstante, la arquitectura permite reformatearla de manera transparente en NTFS, ext4, XFS, Btrfs o UDF. En esta partición, el usuario almacena directamente las imágenes de disco y sus datos personales sin restricciones de nomenclatura jerárquica.
2. **Partición 2 (`VTOYEFI` — Gestor de arranque del sistema):** Es una partición reservada de tamaño fijo (típicamente de 32 megabytes), formateada estrictamente en sistema de archivos FAT (FAT16 o FAT32). En esquemas GPT, se registra con el identificador formal de partición ESP, mientras que en esquemas MBR se codifica con el tipo hexadecimal `0xEF`. Aloja los archivos ejecutables del cargador GNU GRUB modificado, los binarios de firmware para arquitecturas x86_64 e IA-32 (`bootx64.efi`, `bootia32.efi`), controladores del sistema de archivos y módulos de descompresión en memoria.

La decisión arquitectónica de ubicar la partición de datos como Partición 1 y la partición del sistema como Partición 2 responde a una necesidad crítica de compatibilidad con sistemas operativos huésped heredados (Ventoy Project, 2024a). En versiones anteriores a Windows 10 (específicamente versiones previas a la actualización 1703), el subsistema de almacenamiento de Windows trataba a las memorias USB con el descriptor de medio extraíble (*removable media bit*) montando de manera exclusiva la primera partición del medio y ocultando o ignorando cualquier partición subsecuente. Si la partición EFI se hubiese ubicado en la Partición 1, el usuario común no habría podido acceder al volumen de almacenamiento para copiar o eliminar imágenes ISO desde un equipo Windows. Al colocar el volumen de datos en la Partición 1, cualquier sistema operativo monta automáticamente el almacenamiento disponible, manteniendo la Partición 2 protegida de modificaciones accidentales o formateos accidentales por parte del usuario.

2.1.2. Mecanismos de intercepción y enlace dinámico de imágenes

La operación de Ventoy se fundamenta en un proceso de intercepción continua (*hooking*) que opera en dos niveles sucesivos: a nivel de firmware durante la fase de prearranque y a nivel de núcleo durante la inicialización del sistema operativo huésped:

- **Enumeración dinámica:** Durante la secuencia de inicio, el cargador de Ventoy en la Partición 2 toma el control del procesador, explora recursivamente el árbol de directorios de la Partición 1 en busca de archivos con extensiones reconocidas y construye en tiempo real un menú interactivo en memoria RAM. Este mecanismo prescinde de scripts de configuración manuales o reconstrucciones de menú cada vez que se agrega o elimina un sistema operativo.
- **Intercepción en modo BIOS (INT 13h Hooking):** Al seleccionar una imagen ISO en una máquina con BIOS tradicional, el módulo especializado de Ventoy (`vtoy.mod`) mapea los sectores físicos contiguos o fragmentados del archivo ISO sobre la partición exFAT/NTFS. Posteriormente, engancha la interrupción de lectura de disco de la BIOS (INT 13h) en la tabla de vectores de interrupción (IVT) y el área de datos de BIOS extendida (EBDA). A partir de ese instante, cualquier llamada de lectura emitida por el cargador del sistema operativo hacia la unidad de disco virtual emulada es interceptada y traducida matemáticamente a los sectores LBA reales donde reside el archivo ISO dentro de la memoria USB.
- **Intercepción en modo UEFI (UEFI Protocol Hooking):** En plataformas modernas, Ventoy intercepta los protocolos estándar de entrada/salida de bloques del firmware (`EFI_BLOCK_IO_PROTOCOL` y `EFI_DISK_IO_PROTOCOL`), así como el protocolo de sistema de archivos simple (`EFI_SIMPLE_FILE_SYSTEM_PROTOCOL`) (Unified Extensible Firmware Interface Forum, 2024). Ventoy registra un manejador de dispositivo virtual (*virtual device handle*) que expone la imagen ISO ante el firmware como si se tratase de una unidad física de disco óptico CD/DVD conectada al bus.
- **Transición e inyección en el espacio de usuario temprano:** Un desafío fundamental en la emulación de medios de arranque radica en que, en cuanto el kernel de un sistema operativo (Linux o Windows) toma el control, inicializa sus propios controladores de dispositivo nativos (controladores USB xHCI/EHCI, SATA AHCI o NVMe) y abandona por completo los servicios de BIOS o UEFI. En un esquema de emulación simple, el kernel perdería el acceso a la unidad virtualizada y abortaría la carga con errores de medio ausente. Ventoy soluciona este conflicto inyectando código en la fase previa a la ejecución del núcleo:
  - *En distribuciones GNU/Linux:* Ventoy inyecta un módulo auxiliar (`ventoy.cpio`) en la cadena del `initramfs` o modifica la línea de comandos de inicio. Durante la ejecución de `/init`, este script detecta la presencia del medio Ventoy, monta la imagen ISO seleccionada mediante un dispositivo de bucle (`/dev/loopX`) y crea enlaces simbólicos o montajes de unión sobre los directorios esperados por el instalador (`/cdrom`, `/run/live` o `/casper`).
  - *En Microsoft Windows:* Ventoy inyecta un controlador virtual en la memoria física y altera dinámicamente los parámetros de inicialización del entorno WinPE mediante directivas de respuesta desatendida (`unattend.xml`), garantizando que el instalador de Windows localice el contenedor `install.wim` dentro de la imagen ISO virtualizada sin demandar la inserción de discos ópticos.

2.2. Contraste frente a herramientas alternativas

Para evaluar la eficiencia técnica de Ventoy, es indispensable contrastar su arquitectura frente a las soluciones de despliegue y arranque múltiple tradicionalmente empleadas en el ámbito de la administración de sistemas: Rufus, YUMI y el despliegue manual basado en GNU GRUB con el módulo `loopback`.

2.2.1. Rufus y la dicotomía entre modos ISO y DD

Rufus, desarrollada por Pete Batard, es una de las utilerías más difundidas para la creación de medios de instalación de un único sistema operativo (Batard, 2024). Su diseño se enfoca en la optimización de velocidad de escritura y la compatibilidad con tecnologías recientes (como la omisión de requisitos de TPM 2.0 y Secure Boot en Windows 11 o el despliegue de *Windows To Go*). Rufus ofrece dos modos de preparación mutuamente excluyentes:

1. **Modo ISO (Extracción lógica y formateo):** Rufus particiona el dispositivo USB con una tabla MBR o GPT, lo formatea con un sistema de archivos estándar (FAT32 o NTFS) y extrae físicamente el contenido de la imagen ISO archivo por archivo dentro del medio. Posteriormente, instala un cargador específico (Syslinux para BIOS o archivos de arranque para UEFI). En situaciones donde el archivo de instalación de Windows (`install.wim`) excede los 4 gigabytes y la placa exige arranque UEFI bajo FAT32, Rufus implementa la arquitectura `UEFI:NTFS`, creando una partición FAT secundaria diminuta con un controlador de sistema de archivos NTFS para permitir que el firmware UEFI cargue el instalador desde la partición principal formateada en NTFS.
2. **Modo DD (Copia física bloque a bloque):** Aplicable a imágenes híbridas (*ISOHybrid*), Rufus realiza una escritura cruda sector a sector sobre el dispositivo de almacenamiento masivo, calcando fielmente la estructura del archivo ISO 9660. Este método garantiza un arranque idéntico al medio original, pero destruye de manera irreversible cualquier partición preexistente y deja el dispositivo en estado de sólo lectura o con espacio no asignado inaccesible para el almacenamiento convencional.

La limitación estructural insalvable de Rufus radica en que es una herramienta de instalación unitaria: no admite albergar más de un sistema operativo simultáneamente. Cada vez que se requiere cambiar de distribución o actualizar una imagen, la unidad USB debe formatearse y reescribirse por completo.

2.2.2. YUMI y el encadenamiento de cargadores propietarios

YUMI (*Your Universal Multiboot Installer*), desarrollada por Pen Drive Linux, constituyó una de las primeras herramientas exitosas para consolidar múltiples distribuciones en una unidad USB (Pen Drive Linux, 2024). Su arquitectura se basa en un esquema de encadenamiento de cargadores: instala Syslinux como gestor de arranque primario en plataformas BIOS y ramas secundarias de GRUB o GRUB4DOS para gestionar las entradas de menú.

A pesar de su capacidad para albergar diversas imágenes, YUMI presenta deficiencias operativas considerables:

- **Dependencia de configuraciones estáticas:** YUMI no emula discos al vuelo; requiere que sus desarrolladores mantengan una base de datos de configuraciones preprogramadas con los nombres exactos de los núcleos, archivos de disco RAM y parámetros de línea de comandos de cada versión de cada distribución admitida. Si una distribución modifica la estructura interna de sus directorios en un lanzamiento reciente, YUMI falla hasta que la herramienta se actualice con un script adaptado.
- **Restricciones de fragmentación y formato:** Las variantes tradicionales de YUMI demandan el formateo del medio en FAT32, lo que genera incompatibilidad con imágenes de Windows cuyos archivos WIM superan los 4 gigabytes, requiriendo operaciones complejas de particionado secundario o división de archivos en contenedores SWM (*split WIM*).

2.2.3. Configuración manual de GNU GRUB con loopback

La alternativa de bajo nivel consiste en instalar manualmente el cargador GNU GRUB2 en el sector de arranque de la unidad USB y redactar de forma artesanal las directivas de inicio dentro del archivo `grub.cfg` empleando el módulo `loopback` (Free Software Foundation, 2024a). Mediante instrucciones como `loopback loop /imagen.iso` y fijando el volumen raíz con `set root=(loop)`, el administrador instruye a GRUB para acceder al árbol de directorios contenido dentro de la imagen ISO sin descomprimirla en el disco.

No obstante, esta metodología demanda un mantenimiento intensivo e impone severas restricciones funcionales:

- Cada distribución Linux requiere parámetros de arranque del kernel completamente dispares para localizar la imagen ISO montada en bucle (por ejemplo, directivas como `iso-scan/filename` en Debian y Ubuntu, frente a `img_dev` con `img_loop` en Arch Linux, o la directiva `rd.live.image` en la familia Red Hat/Fedora).
- Resulta enteramente inoperante para el arranque de imágenes de instalación de Microsoft Windows o entornos WinPE. Dado que el cargador de Windows (`bootmgr`) y el kernel NT no incorporan controladores nativos para interpretar dispositivos de bucle generados por GRUB, el proceso colapsa de forma prematura solicitando controladores ópticos inexistentes.

La Tabla 2 presenta una síntesis comparativa entre estas tecnologías, evidenciando las ventajas estructurales de la emulación dinámica en medios de arranque múltiple.

Tabla 2: Comparativa técnica de herramientas y metodologías para medios de arranque USB.

Parámetro	Herramientas tradicionales (Rufus / YUMI)	Emulación dinámica (Ventoy)
Capacidad multiboot	Inexistente en Rufus (unitario); soportado en YUMI mediante encadenamiento estático de cargadores.	Nativa y escalable; admite decenas de imágenes ISO heterogéneas coexistiendo simultáneamente.
Manejo de archivos de imagen	Extracción física de archivos (modo ISO) o clonación destructiva sector a sector (modo DD).	Enlace directo de archivos de imagen (`.iso`, `.wim`) sin descompresión ni alteración física.
Actualización de sistemas	Requiere reformatear la unidad (Rufus) o ejecutar un asistente para reescribir scripts (YUMI).	Operación de copiado y borrado estándar mediante el administrador de archivos del sistema operativo.
Mecanismo de emulación	Nulo; depende de la estructura plana extraída en el sistema de archivos del pendrive.	Intercepción de bajo nivel: INT 13h en BIOS y protocolos de bloque EFI en UEFI, con inyección en el kernel.
Compatibilidad multiplataforma	Compleja en YUMI (diferenciación entre ramas Legacy y UEFI); asistida en Rufus (UEFI:NTFS).	Soporte dual transparente: arranca en BIOS x86, UEFI IA-32 y UEFI x86_64 desde el mismo medio.

3. Análisis técnico de los entornos integrados

Un medio de arranque múltiple profesional debe estructurarse a partir de un acervo heterogéneo de sistemas operativos y entornos de diagnóstico que respondan a distintos requerimientos de ingeniería: rescate de emergencia en plataformas heredadas, análisis de bajo nivel de hardware, auditoría de seguridad informática, administración de infraestructura de servidores y despliegue masivo de estaciones de trabajo cliente. Cada uno de los entornos integrados en la memoria USB presenta decisiones de diseño arquitectónicas dispares en cuanto al núcleo del sistema operativo, el subsistema de inicio de servicios, la gestión de memoria virtual y la abstracción de almacenamiento en vivo.

3.1. Entornos de diagnóstico y rescate: Hiren's BootCD 15.2 y Knoppix 8.6

Los entornos de diagnóstico están orientados a operar de manera independiente a los sistemas operativos instalados en los discos locales de la máquina anfitriona, permitiendo intervenir plataformas comprometidas por fallos de hardware, corrupción de sistemas de archivos o infecciones de *malware*.

3.1.1. Hiren's BootCD 15.2 y la suite de utilerías en modo real

Hiren's BootCD versión 15.2 representa la consolidación histórica de las herramientas de diagnóstico para la arquitectura x86 de 32 bits (IA-32) y plataformas gobernadas por BIOS tradicional (Hiren's BootCD Team, 2012). Su arquitectura de arranque se apoya en los gestores ISOLINUX y GRUB4DOS, los cuales bifurcan el flujo de control hacia tres entornos operativos diferenciados:

1. **Mini Windows XP:** Es una edición compacta y personalizada del núcleo NT 5.1 (Windows XP), diseñada para cargarse íntegramente en la memoria RAM mediante el controlador de disco en memoria `ramdisk.sys`. Al no depender del disco duro del equipo, Mini Windows XP proporciona un entorno gráfico Win32 estable desde el cual ejecutar utilerías de recuperación de particiones, clonación de archivos de usuario y desinfección forense.
2. **Entorno DOS extendido (DOS BootCD):** Basado en FreeDOS y MS-DOS, este subsistema opera en modo real de 16 bits asistido por controladores de memoria extendida y expandida (`HIMEM.SYS` y `EMM386.EXE`), junto con controladores de interfaz de programación de software avanzada (ASPI) para acceder a controladoras SCSI y SATA. Su propósito esencial es ejecutar utilerías que requieren comunicación directa con los puertos de entrada/salida (I/O) y registros del procesador sin la interferencia de las capas de abstracción o la paginación de memoria de un sistema operativo multitarea.
3. **Entorno de utilerías Linux:** Un núcleo Linux monolítico mínimo emparejado con herramientas de consola para clonación a nivel de bloques y particionado no destructivo.

Entre las herramientas de bajo nivel integradas en Hiren's BootCD destacan:

- **Memtest86+:** Herramienta de verificación de la memoria física que opera en modo protegido de 32 bits sin cargar ningún sistema operativo subyacente. Ejecuta algoritmos de prueba de marcha (tales como March C-) y patrones de inversión de bits sobre las matrices de memoria DRAM física, permitiendo diagnosticar con precisión celdas

de memoria defectuosas, fallos de direccionamiento en el controlador de memoria y problemas de estabilidad térmica o temporización en los módulos de RAM.

- **MHDD y Victoria:** Utilerías de diagnóstico de almacenamiento masivo que se comunican directamente con los puertos ATA/SATA primarios y secundarios de la placa base (registros de puertos I/O `0x1F0` y `0x170`). Realizan pruebas de lectura sector por sector midiendo la latencia de respuesta en milisegundos para detectar sectores débiles antes de que se transformen en sectores defectuosos irreparables, además de leer los atributos de diagnóstico SMART del disco duro.
- **Offline NT Password & Registry Editor (`chntpw`):** Utilería de rescate basada en Linux que permite manipular sin conexión la base de datos de cuentas locales de Windows (SAM, *Security Account Manager*), ubicada en el archivo `config\SAM` de la partición del sistema. La herramienta analiza la estructura binaria de bloques de la colmena del registro, limpia el hash criptográfico NT de contraseñas olvidadas de cuentas de administrador y desactiva banderas de bloqueo de cuenta sin requerir descifrado de fuerza bruta.

3.1.2. Knoppix 8.6: Arquitectura híbrida y eliminación de systemd

Knoppix 8.6, desarrollada por Klaus Knopper, constituye una distribución en vivo de referencia construida sobre la base de Debian Stable (Buster) con paquetes seleccionados de las ramas de prueba para soporte de hardware moderno (Knopper, 2019). Knoppix destaca por dos decisiones arquitectónicas de ingeniería de gran relevancia:

- **Arquitectura híbrida de núcleo y espacio de usuario:** El medio de Knoppix 8.6 incorpora un núcleo Linux versión 5.3 compilado en dos variantes: 64 bits (`x86_64`) y 32 bits (`i386`). Durante la fase de inicio, el cargador inspecciona las banderas de la CPU en busca de la instrucción de modo largo (*long mode*). Si el procesador soporta 64 bits, Knoppix arranca automáticamente el núcleo de 64 bits, pero ejecuta un espacio de usuario compilado íntegramente en 32 bits. Esta combinación confiere compatibilidad binaria absoluta con software de diagnóstico heredado a la vez que permite direccionar la totalidad de la memoria RAM del equipo (superando holgadamente el límite de 4 gigabytes de IA-32 sin sufrir las penalizaciones de rendimiento del direccionamiento físico extendido, PAE) y ejecutar aplicaciones de 64 bits cuando sea requerido.
- **Reemplazo de systemd por SysVinit y elogind:** A partir de la versión 8.5 y consolidado en la 8.6, Knoppix eliminó por completo el gestor `systemd` como proceso PID 1 (Knopper, 2019). En su lugar, el sistema emplea el esquema clásico de inicialización SysVinit con scripts secuenciales bajo `/etc/init.d/`, complementado con el demonio independiente `elogind` para la administración de sesiones de usuario y eventos de energía (reinicio y apagado). Esta arquitectura fue adoptada para acelerar los tiempos de inicio desde medios extraíbles lentos (memorias USB y discos ópticos) y garantizar que los scripts de detección de hardware operen de forma lineal y determinista, evitando dependencias circulares y bloqueos de servicios concurrentes.
- **Sistema de archivos comprimido cloop y unión AUFS:** La totalidad del sistema operativo (que equivale a más de 10 gigabytes de software descomprimido) se aloja en un único archivo de aproximadamente 4.5 gigabytes gracias al controlador de dispositivo de bucle comprimido `cloop` (Knopper, 2024). Mediante el sistema de archivos de unión AUFS, Knoppix superpone un sistema de archivos volátil en memoria RAM (`tmpfs`) sobre la capa de sólo lectura de `cloop`, creando una ilusión perfecta de un

sistema operativo de lectura y escritura.

- **Autodetección mediante** `knoppix-autoconfig`**:** Al encender el equipo, un motor de scripts de bajo nivel sondea los buses PCI, PCIe y USB mediante la interfaz `/sys`, identifica los adaptadores de vídeo (Intel, AMD, Nvidia), carga los controladores de aceleración directa en el kernel (DRM/KMS) y configura de manera autónoma el servidor X.Org con el entorno de escritorio ligero LXDE.

3.2. Sistemas GNU/Linux para servidor y escritorio (32 y 64 bits)

El medio multiboot integra asimismo dos distribuciones GNU/Linux de propósito específico: un sistema operativo optimizado para infraestructura de red y servicios (Linux Server), y una distribución orientada a estaciones de trabajo y auditoría de seguridad (Kali Linux / Linux Desktop).

3.2.1. Arquitectura de Linux Server y gestión de servicios

Las distribuciones de servidor, como Ubuntu Server 22.04 LTS o Debian Server (Canonical Ltd., 2024), están diseñadas para maximizar el rendimiento acumulado (*throughput*) y la confiabilidad operativa en tareas de procesamiento ininterrumpido. A diferencia de las distribuciones de escritorio, el núcleo de Linux para servidores implementa directivas de compilación y planificación especializadas:

1. **Políticas de planificación del kernel:** El planificador de procesos opera bajo el esquema de preempción voluntaria (directiva del kernel `PREEMPT_VOLUNTARY`) o sin preempción forzada (`PREEMPT_NONE`). Esta configuración reduce al mínimo la frecuencia de cambios de contexto involuntarios entre procesos, favoreciendo a los servicios que realizan cómputo intensivo o transacciones concurrentes de base de datos a expensas de la latencia interactiva de interfaces visuales (Stallings, 2018, §3.2, p. 132).
2. **Frecuencia de reloj del sistema:** El temporizador del kernel se configura a frecuencias reducidas (típicamente 100 Hz o 250 Hz mediante `CONFIG_HZ=250`), lo que disminuye drásticamente el número de interrupciones de reloj procesadas por la CPU por segundo y permite que los núcleos de procesamiento permanezcan en estados de ejecución continua de hilos de servidor.
3. **Soporte para topologías NUMA y SMP:** Incorpora algoritmos avanzados de asignación de memoria basados en la topología de nodos de memoria no uniforme (NUMA, *non-uniform memory access*), asegurando que los hilos de ejecución reserven páginas en los bancos de memoria físicamente más cercanos al zócalo del procesador donde están asignados.

El espacio de usuario de Linux Server es administrado por `systemd` como proceso PID 1, estructurando los servicios mediante unidades (*units*), temporizadores (*timers*) y activación basada en sockets. Los procesos se confinan de manera estricta mediante grupos de control jerárquicos (*cgroups v2*), permitiendo fijar límites deterministas de cuotas de CPU, memoria física y ancho de banda de E/S por servicio. Finalmente, Linux Server carece intencionalmente de servidor de despliegue gráfico (sin X11 ni Wayland); su administración se efectúa exclusivamente a través de consolas virtuales TTY o sesiones remotas cifradas mediante el protocolo SSH, reduciendo la superficie de ataque y manteniendo el consumo basal de memoria RAM en reposo por debajo de los 512 megabytes.

3.2.2. Kali Linux: Entorno de auditoría y modo forense

Para tareas de diagnóstico de seguridad, evaluación de vulnerabilidades y administración de red, el pendrive incorpora Kali Linux, una distribución basada en Debian Testing mantenida por Offensive Security (Offensive Security, 2024). La arquitectura en vivo de Kali Linux se estructura mediante los scripts del espacio de usuario temprano `live-boot` y un sistema de archivos comprimido con el algoritmo SquashFS.

Kali Linux implementa dos mecanismos operacionales de alta relevancia técnica:

- **Modo Forense (*Forensic Mode*):** Es una modalidad de arranque especializada para informática forense y análisis de incidentes. Al seleccionar esta opción en el menú de inicio, el kernel y los scripts de `initramfs` configuran el sistema operativo para:
  - Desactivar por completo el montaje automático de cualquier unidad de disco duro interno, partición de estado sólido o medio USB conectado a la computadora anfitriona.
  - Bloquear de forma estricta la inicialización de particiones de intercambio (*swap*) presentes en los discos de la máquina, impidiendo que páginas de memoria RAM del sistema forense se escriban accidentalmente sobre el medio inspeccionado.
  - Garantizar la integridad probatoria y la cadena de custodia de la evidencia digital, operando en modo estrictamente no invasivo.
- **Persistencia cifrada mediante LUKS:** A diferencia de los medios en vivo convencionales que pierden sus cambios al reiniciar, Kali Linux permite configurar una partición de persistencia sobre el pendrive cifrada con LUKS (*Linux Unified Key Setup*). Durante el inicio temprano, el script de `initramfs` solicita la contraseña simétrica, desbloquea el contenedor criptográfico mediante el subsistema `dm-crypt` y monta el volumen resultante como capa superior de escritura mediante OverlayFS.

El entorno de usuario integra el gestor de escritorio XFCE y proporciona acceso nativo a controladores de red con soporte para inyección de paquetes en crudo (*raw packet injection*) y conmutación de tarjetas inalámbricas a modo monitor, indispensables para auditorías de telecomunicaciones.

3.3. Entorno de instalación de Microsoft Windows 10 (WinPE, 32 y 64 bits)

La inclusión del medio de instalación de Microsoft Windows 10 en versiones de 32 bits (IA-32) y 64 bits (x86_64) dentro de la unidad multiboot atiende a la necesidad de implementar tareas de despliegue, recuperación del almacén BCD y mantenimiento de estaciones de trabajo cliente (Microsoft Corporation, 2024b).

3.3.1. Arquitectura del núcleo híbrido NT 10.0 y el subsistema WinPE

El núcleo de Windows 10 se fundamenta en la arquitectura NT 10.0, clasificada como un núcleo híbrido que combina la modularidad de un microkernel con el rendimiento de un núcleo monolítico (Stallings, 2018, §2.4, p. 75). Sus componentes centrales se estructuran en capas concéntricas:

1. **Capa de Abstracción de Hardware (HAL):** Oculta las disparidades de los controladores de interrupción programable (APIC), temporizadores de plataforma y controladores de bus de la placa base, ofreciendo una interfaz uniforme al resto del sistema operativo.

2. **Microkernel NT:** Gestiona la planificación de hilos de ejecución con prioridades preventivas, el despacho de interrupciones de hardware y la sincronización entre procesadores en entornos multinúcleo.
3. **Servicios Ejecutivos (`ntoskrnl.exe`):** Residen en el espacio de memoria del núcleo e integran los administradores fundamentales: el Administrador de Procesos e Hilos, el Administrador de Memoria Virtual (VMM), el Administrador de Seguridad (SRM), el Administrador de Entrada/Salida y el subsistema gráfico `win32k.sys`.

Durante la fase de instalación o recuperación, el sistema no ejecuta la instalación completa de Windows, sino el entorno de preinstalación de Windows (WinPE 10.0) alojado dentro de una imagen de contenedor en formato WIM (*Windows Imaging Format*) denominada `boot.wim` (Microsoft Corporation, 2024c). El formato WIM utiliza compresión basada en archivos (algoritmos LZX o XPRESS) con tecnología de almacenamiento monoinstancia (*single-instance storage*), lo que permite que múltiples versiones del sistema operativo (Home, Pro, Enterprise) compartan los mismos archivos sin duplicación física en el disco.

El asistente de instalación (`setup.exe`) interactúa con las APIs de bajo nivel de Windows para preparar el almacenamiento secundario: invoca el motor de particionado de `diskpart` para estructurar la unidad de disco destino (creando en UEFI la partición ESP en FAT32, la partición reservada de Microsoft MSR de 16 megabytes y la partición principal de sistema en NTFS), extrae y aplica la imagen de `install.wim` mediante la API WIMGAPI, inyecta los controladores críticos del equipo y genera las estructuras del cargador en la partición ESP invocando la herramienta `bcdboot.exe`.

3.3.2. Divergencias arquitectónicas entre 32 bits y 64 bits

La coexistencia de imágenes de instalación de Windows 10 de 32 bits y 64 bits evidencia transformaciones profundas en el manejo de memoria y las políticas de seguridad del procesador:

- **Espacio de direccionamiento virtual:** En arquitecturas IA-32 de 32 bits, el procesador está restringido a direccionar un espacio virtual de 2^{32} bytes (4 gigabytes). Por defecto, el Administrador de Memoria de Windows divide este espacio de forma equitativa: 2 gigabytes para el espacio de usuario privado de cada proceso y 2 gigabytes para el espacio del núcleo, lo que genera una severa contención de memoria en el fondo no paginado (*non-paged pool*) al gestionar controladores complejos. En contraparte, en la arquitectura x86_64 (AMD64) con direccionamiento canónico de 48 bits, cada proceso de usuario dispone de un espacio virtual privado de 128 terabytes, y el núcleo cuenta con otros 128 terabytes dedicados, eliminando las restricciones de memoria virtual.
- **Estructura de registros de la CPU:** La arquitectura de 64 bits duplica el número de registros de propósito general (de 8 a 16 registros, incorporando de R8 a R15) y amplía su ancho de banda de 32 a 64 bits (RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP), además de duplicar el banco de registros vectoriales de XMM0–XMM7 a XMM0–XMM15. Esto permite que las llamadas a funciones utilicen la convención de paso de parámetros por registros rápidos (*fastcall*) en lugar de recurrir continuamente a operaciones de apilamiento en memoria RAM.
- **Mecanismos de seguridad exclusivos de 64 bits:**
  1. *Firma obligatoria de controladores (KMCS, Kernel Mode Code Signing):* La edición de 64 bits prohíbe terminantemente la carga de controladores de dispositivo en modo núcleo que no cuenten con una firma criptográfica emitida por una enti-

dad certificadora autorizada por Microsoft, erradicando la inserción de *rootkits* a nivel de kernel.

2. *Protección contra parches del núcleo (KPP / PatchGuard):* Mecanismo de supervisión que verifica periódicamente la integridad de las estructuras críticas del sistema (tablas SSDT, IDT, GDT y el propio código de `ntoskrnl.exe`). Si un proceso o controlador intenta interceptar llamadas al sistema alterando estas tablas, el microkernel aborta inmediatamente la ejecución con una comprobación de error de parada (*BugCheck 0x109 CRITICAL_STRUCTURE_CORRUPTION*).
3. *Prevención de ejecución de datos forzada por hardware (DEP / bit NX):* La MMU del procesador marca las páginas de memoria de la pila y el montículo como no ejecutables, neutralizando ataques de desbordamiento de búfer.

La Tabla 3 resume las propiedades arquitectónicas de todos los entornos integrados en el medio multiboot.

Tabla 3: Matriz comparativa de sistemas operativos y entornos integrados en el medio multiboot.

Entorno / Sistema	**Arquitectura y Núcleo**	**Esquema de Inicio (PID 1)**	**Propósito y Caso de Uso**
Hiren's BootCD 15.2	32 bits (IA-32); núcleos híbridos NT 5.1 (MiniXP) y FreeDOS.	ISOLINUX / GRUB4DOS hacia ramdisk o modo real.	Diagnóstico de memoria (Memtest), reparación de discos (MHDD) y restablecimiento SAM.
Knoppix 8.6	Híbrida: kernel Linux 5.3 (x86_64) con espacio de usuario de 32 bits.	SysVinit secuencial complementado con elogind.	Sistema en vivo portátil; compresión cloop, soporte de persistencia y hardware antiguo.
Linux Server (Ubuntu / Debian)	64 bits (x86_64); kernel Linux monolítico con preempción voluntaria.	systemd basado en sockets, units y cgroups v2.	Servicios de red, infraestructura de virtualización y bases de datos en modo consola TTY.
Kali Linux	64 bits (x86_64); kernel Linux monolítico con liveboot y SquashFS.	systemd; soporte para scripts de modo forense.	Auditorías de penetración, análisis forense no invasivo y persistencia cifrada LUKS.
Windows 10 (32 y 64 bits)	32/64 bits dual; núcleo híbrido Windows NT 10.0 (WinPE 10.0).	bootmgr con BCD hacia imagen boot.wim en RAM.	Despliegue de estaciones de trabajo, gestión WIM/DISM y reparación del cargador BCD.

4. Metodología de preparación y despliegue del medio multiboot

La construcción de un medio de almacenamiento extraíble de arranque múltiple exige una metodología rigurosa que contemple la inspección del dispositivo físico de bloques, la selección de la tabla de particiones adecuada, el despliegue del cargador de arranque y la gestión de parámetros avanzados de configuración y seguridad. La correcta ejecución de estas fases garantiza que la unidad sea reconocida tanto por placas base antiguas gobernadas por BIOS como por estaciones de trabajo y servidores contemporáneos que exigen arranque nativo bajo UEFI con validación criptográfica de firmas.

4.1. Preparación del dispositivo y despliegue de Ventoy mediante consola

El despliegue de la infraestructura de arranque se realiza en un entorno GNU/Linux utilizando utilerías del espacio de usuario y scripts de bajo nivel provistos por el proyecto Ventoy (Ventoy Project, 2024a). Este procedimiento demanda la identificación unívoca del nodo de dispositivo de bloques correspondiente al medio USB para evitar la sobreescritura accidental de los discos duros o unidades de estado sólido del sistema operativo anfitrión.

4.1.1. Identificación de bloques y análisis de capacidad requerida

Antes de conectar el medio USB, se consulta la topología de almacenamiento existente mediante el comando `lsblk`. Al insertar la unidad, se repite la consulta evaluando las columnas de transporte (`TRAN`), tamaño (`SIZE`) y puntos de montaje (`MOUNTPOINTS`):

```
lsblk -o NAME,SIZE,TYPE,TRAN,FSTYPE,LABEL,MOUNTPOINTS
fdisk -l /dev/sdb
```

La salida de estos comandos permite confirmar que el nodo asignado (por ejemplo, `/dev/sdb`) posee el atributo de transporte `usb` y carece de particiones del sistema anfitrión montadas. En caso de que el entorno de escritorio haya montado automáticamente particiones preexistentes, es obligatorio desmontarlas antes de manipular la tabla de particiones:

```
sudo umount /dev/sdb* 2>/dev/null || true
sudo wipefs -a /dev/sdb
```

El uso de `wipefs -a` destruye firmas residuales de sistemas de archivos antiguos (tales como números mágicos de ext4, NTFS o descriptores de volumen ISO 9660), previniendo inconsistencias en los controladores del núcleo al releer la tabla de particiones.

El dimensionamiento de la unidad responde a la suma de las imágenes de disco que conformarán el catálogo del proyecto:

- Hiren's BootCD 15.2: ≈ 600 MB.
- Ubuntu Server 22.04 LTS (64 bits): ≈ 1.8 GB.
- Knoppix 8.6 Live DVD: ≈ 4.5 GB.
- Kali Linux 2024 Live (64 bits): ≈ 3.8 GB.
- Microsoft Windows 10 (instaladores de 32 y 64 bits): ≈ 5.8 GB.
- Espacio reservado para persistencia y almacenamiento de usuario: ≈ 4.0 GB.

El volumen total indispensable asciende a aproximadamente 20.5 gigabytes, por lo que se requiere una memoria USB con una capacidad nominal mínima de 32 gigabytes (o preferentemente 64 gigabytes para admitir imágenes adicionales y registros de auditoría sin saturación de bloques).

4.1.2. Instalación de Ventoy con particionado GPT y Secure Boot

El paquete oficial de Ventoy para entornos Linux se distribuye como un archivo comprimido que integra los binarios del cargador para x86, IA-32 y ARM, junto con el script de instalación en consola `Ventoy2Disk.sh`. Para garantizar la máxima interoperabilidad con hardware contemporáneo, la instalación se parametriza mediante las siguientes banderas:

- `-i`: Inicialización e instalación destructiva primaria sobre el dispositivo destino.
- `-g`: Selección explícita de la tabla de particiones GUID (GPT), sustituyendo el esquema MBR por defecto para permitir compatibilidad plena con arquitecturas UEFI nativas y particiones ESP estándar.
- `-s`: Habilitación obligatoria del soporte para Secure Boot, integrando el gestor *shim* y certificados criptográficos en la partición del sistema.
- `-r <MB>`: Opción para reservar un espacio sin particionar al final del medio (por ejemplo, `-r 4096` para reservar 4 GB destinados a particiones secundarias dedicadas).

La ejecución del comando se ilustra en el siguiente bloque de terminal:

```
tar -xzvf ventoy-1.0.99-linux.tar.gz
cd ventoy-1.0.99
sudo ./Ventoy2Disk.sh -i -g -s /dev/sdb
```

El script solicita confirmación doble para prevenir errores operativos y procede a:

1. Escribir el MBR protector en el LBA 0 y generar la cabecera GPT primaria en el LBA 1.
2. Crear la Partición 2 (`VTOYEFI`) al final del medio como una partición de 32 megabytes, asignándole el GUID de sistema EFI (`EF00`) y formateándola en FAT16 con los binarios `bootx64.efi`, `bootia32.efi` y el cargador de GRUB2 firmado.
3. Crear la Partición 1 (`Ventoy`) abarcando el espacio restante del medio y formateándola por defecto con el sistema de archivos exFAT.
4. Generar la cabecera GPT secundaria y la matriz de respaldo en los últimos 33 sectores físicos de la unidad USB.

La comprobación posterior con `lsblk -f /dev/sdb` corrobora la creación del volumen `/dev/sdb1` (etiqueta `Ventoy`, exFAT) y el volumen `/dev/sdb2` (etiqueta `VTOYEFI`, vfat).

4.2. Transferencia de imágenes de disco y configuración de parámetros

Una vez desplegada la estructura de arranque, la unidad USB se encuentra lista para recibir las imágenes de los sistemas operativos. A diferencia de las metodologías convencionales, no se requiere ningún software adicional para quemar o extraer las imágenes: la transferencia se efectúa como una operación estándar de copia de archivos a nivel de sistema de archivos.

4.2.1. Organización jerárquica y transferencia sincronizada

Se monta la partición principal de datos (`/dev/sdb1`) en un directorio local del sistema anfitrión. Aunque Ventoy realiza una búsqueda recursiva automática de imágenes en cualquier nivel de directorios, en ingeniería de sistemas se adopta una taxonomía modular que facilite el mantenimiento y la segmentación funcional:

```
sudo mount /dev/sdb1 /mnt/ventoy
sudo mkdir -p /mnt/ventoy/ISO/{Diagnostic,Live,Server,Security,Windows}
```

La copia de imágenes de gran tamaño (superiores a 4 GB) a través del bus USB está expuesta a retardos por almacenamiento en búferes volátiles de la memoria RAM del sistema operativo. Si el usuario desconecta el dispositivo antes de que el kernel descargue completamente las páginas sucias (*dirty pages*) al almacenamiento flash, los archivos ISO sufrirán corrupciones silenciosas de sectores. Por ello, la transferencia se ejecuta mediante `rsync` con parámetros de progreso, seguida de la invocación explícita de la llamada al sistema `sync`:

```
sudo rsync -avh -progress /origen/Hirens.BootCD.15.2.iso /mnt/ventoy/ISO/Diagnostic/
sudo rsync -avh -progress /origen/knoppix-8.6.iso /mnt/ventoy/ISO/Live/
sudo rsync -avh -progress /origen/ubuntu-22.04-server.iso /mnt/ventoy/ISO/Server/
sudo rsync -avh -progress /origen/kali-linux-2024.iso /mnt/ventoy/ISO/Security/
sudo rsync -avh -progress /origen/Win10_Spanish_x64.iso /mnt/ventoy/ISO/Windows/
sync
```

Finalizado el vaciado físico de memoria con `sync`, se calcula la suma criptográfica SHA-256 de cada archivo directamente desde el medio USB montado, contrastándola contra el valor reportado por el proveedor oficial de la imagen para descartar alteraciones por tasas de error de bit (BER) en la interfaz USB:

```
sha256sum /mnt/ventoy/ISO/Security/kali-linux-2024.iso
```

4.2.2. Configuración avanzada mediante directivas ventoy.json

El comportamiento en tiempo de ejecución del gestor Ventoy se personaliza mediante un archivo de configuración en formato JSON denominado `ventoy.json`, el cual debe residir estrictamente en la ruta `/ventoy/ventoy.json` de la Partición 1 (Ventoy Project, 2024b). A través de este archivo se activan módulos de persistencia, control de interfaz y opciones de despliegue desatendido.

Para este proyecto, se implementan tres plugins esenciales:

1. **Plugin de control global (`control`):** Define el tiempo de espera del menú (parámetro `VTOY_MENU_TIMEOUT`) en 15 segundos, fija la resolución gráfica de GOP en 1024×768 y activa la omisión de comprobaciones en Windows.
2. **Plugin de persistencia (`persistence`):** Asocia a la imagen de Kali Linux un contenedor de almacenamiento persistente formateado en ext4 (`kali_persistence.dat`), permitiendo guardar herramientas adicionales, actualizaciones y cambios de configuración entre reinicios sucesivos (Ventoy Project, 2024b).
3. **Plugin de alias de menú (`menu_alias`):** Sustituye los nombres largos y técnicos de los archivos ISO en pantalla por títulos claros y legibles para el operador técnico.

La estructura del archivo `ventoy.json` configurado se presenta a continuación:

```
{
  çontrol": [
    {"VTOY_MENU_TIMEOUT": "15"},
    {"VTOY_DEFAULT_SEARCH_ROOT": /ISO"}
  ],
  "persistence": [
    {
      ïmage": /ISO/Security/kali-linux-2024.iso",
      "backend": /ventoy/persistence/kali_persistence.dat"
    }
  ],
  "menu_alias": [
    {
      ïmage": /ISO/Diagnostic/Hirens.BootCD.15.2.iso",
      .ªlias": "Hiren's BootCD 15.2 (Diagnóstico)"
    },
    {
      ïmage": /ISO/Live/knoppix-8.6.iso",
      .ªlias": "Knoppix 8.6 Live (LXDE / cloop)"
    },
    {
      ïmage": /ISO/Server/ubuntu-22.04-server.iso",
      .ªlias": "GNU/Linux Server (Ubuntu 22.04 LTS)"
    },
    {
      ïmage": /ISO/Security/kali-linux-2024.iso",
      .ªlias": "Kali Linux 2024 (Forense / Pentest)"
    },
    {
      ïmage": /ISO/Windows/Win10_Spanish_x64.iso",
      .ªlias": "Windows 10 (32/64 bits WinPE)"
    }
  ]
}
```

El archivo contenedor de persistencia se genera con la utilería provista por Ventoy (`CreatePersistentImg.sh`), asignándole un tamaño de 4 gigabytes con el sistema de archivos ext4 y la etiqueta reglamentaria `persistence`:

```
sudo sh CreatePersistentImg.sh -s 4096 -t ext4 -l persistence -c kali_persistence.dat
sudo mv kali_persistence.dat /mnt/ventoy/ventoy/persistence/
```

4.2.3. Enrolamiento del certificado MOK para Secure Boot

Cuando la memoria USB se conecta a una computadora que tiene habilitada la tecnología Secure Boot en su firmware UEFI, el cargador de arranque primario de Ventoy no puede ejecutarse directamente si la firma digital de su binario no está preinstalada de fábrica en la base de datos `db` de la placa base (Ventoy Project, 2024a).

Para superar esta restricción sin obligar al usuario a deshabilitar Secure Boot en el menú del firmware, Ventoy implementa el estándar de la industria basado en el cargador intermediario *shim* y la herramienta de gestión de claves del propietario de la máquina (MokManager, `MokManager.efi`):

1. Al iniciar por primera vez en un equipo con Secure Boot activo, el firmware bloquea la ejecución de GRUB y transfiere el control a la pantalla azul de la utilidad MokManager.

2. El usuario selecciona en el menú en pantalla la opción *Enroll key from disk*.
3. Se navega a la partición del sistema `VTOYEFI` y se localiza el certificado público oficial de Ventoy denominado `ENROLL_KEY.CER`.
4. Se selecciona la opción de visualizar la clave para comprobar la huella digital criptográfica y se presiona *Continue*.
5. El sistema solicita la clave de confirmación de enrolamiento (*enrollment password*), cuyo valor predeterminado es `ventoy` (en minúsculas).
6. Se confirma la operación y se selecciona *Reboot*.

Tras reiniciar la máquina, la clave pública de Ventoy queda almacenada de manera permanente en la memoria NVRAM del equipo bajo el registro MOK. En todos los arranques posteriores, el firmware valida automáticamente la firma de Ventoy y permite la ejecución inmediata del menú de inicio y de cualquier imagen ISO cargada sin advertencias de seguridad.

5. Protocolo de verificación y pruebas de operatividad

Para certificar que la memoria USB multiboot cumple con los estándares de ingeniería y responde con fidelidad ante escenarios reales de soporte técnico, administración de servidores y despliegue de estaciones de trabajo, se instrumentó un protocolo riguroso de pruebas funcionales. La verificación comprende la evaluación del menú interactivo, la comprobación de la emulación de medios ópticos, el diagnóstico de bajo nivel del hardware y la validación de la inicialización completa de cada uno de los sistemas operativos integrados bajo diversas configuraciones de firmware y arquitecturas de procesador.

5.1. Entorno de validación y control del menú de arranque

El banco de pruebas se configuró bajo un esquema dual que combina la ejecución sobre hardware físico representativo con pruebas controladas mediante virtualización asistida por hardware.

5.1.1. Infraestructura de pruebas: Hardware real y emulación con QEMU/KVM

La verificación en hardware físico se desarrolló sobre dos plataformas con características complementarias:

1. **Plataforma contemporánea UEFI x86_64:** Estación de trabajo equipada con procesador Intel Core de 64 bits y firmware UEFI clase 3 (InsydeH2O), evaluada tanto en modalidad de arranque seguro (*Secure Boot*) habilitado como con soporte de módulo de compatibilidad heredada (CSM) deshabilitado.
2. **Plataforma heredada BIOS / CSM:** Computadora portátil con procesador multinúcleo x86 y firmware BIOS tradicional de 32/64 bits, orientada a evaluar la conmutación al modo real y la intercepción de interrupciones INT 13h.

Para acelerar la iteración y permitir la inspección exhaustiva de los registros de arranque sin forzar ciclos continuos de reinicio en el equipo de desarrollo, se implementó un entorno de virtualización de alto rendimiento utilizando QEMU (*Quick Emulator*) acoplado al módulo del núcleo KVM (*Kernel-based Virtual Machine*) (QEMU Project, 2024). Mediante el

paso directo del dispositivo físico de bloques (*device passthrough*), QEMU ejecuta el medio USB físico en crudo (`/dev/sdb`), garantizando un comportamiento idéntico al de una máquina física.

Para emular el arranque bajo BIOS tradicional se ejecutó el siguiente comando en consola:

```
sudo qemu-system-x86_64 -enable-kvm -m 4096 -smp 4
  -drive file=/dev/sdb,format=raw,cache=none,if=virtio
  -boot menu=on -vga std
```

Para la validación en modo UEFI nativo con soporte de Secure Boot, se recurrió a los binarios de firmware de código abierto OVMF (*Open Virtual Machine Firmware*), instanciando una copia temporal de la memoria no volátil para preservar las variables NVRAM y los certificados MOK:

```
cp /usr/share/edk2-ovmf/x64/OVMF_VARS.fd /tmp/ovmf_vars.fd
sudo qemu-system-x86_64 -enable-kvm -m 4096 -smp 4
  -drive if=pflash,format=raw,readonly=on,file=/usr/share/edk2-ovmf/x64/OVMF_CODE.fd
  -drive if=pflash,format=raw,file=/tmp/ovmf_vars.fd
  -drive file=/dev/sdb,format=raw,cache=none,if=virtio
  -boot menu=on -vga std
```

5.1.2. Comprobación del menú interactivo de Ventoy

Al encender las máquinas de prueba, el cargador de Ventoy en la Partición 2 tomó el control inmediato de la pantalla:

- **Resolución y modo gráfico:** En UEFI, el protocolo GOP inicializó la pantalla a una resolución limpia de 1024×768 píxeles a 32 bits de profundidad de color; en BIOS, se estableció el modo VBE correspondiente sin distorsión de relación de aspecto.
- **Procesamiento de alias y jerarquía:** El menú principal desplegó correctamente los alias amigables configurados en el archivo `ventoy.json` en lugar de los nombres de archivo brutos. Se verificó que las imágenes ubicadas en subdirectorios dentro de `/ISO/` fuesen listadas de forma unificada.
- **Temporizador y navegación:** La cuenta regresiva del temporizador regulado por `VTOY_MENU_TIMEOUT` inició en 15 segundos y se detuvo automáticamente ante cualquier pulsación de tecla del cursor, permitiendo seleccionar con fluidez cada una de las imágenes.

5.2. Comprobación de utilerías y sistemas de diagnóstico

Se procedió con la ejecución individual de los dos entornos dedicados a la recuperación de emergencia y el mantenimiento de hardware: Hiren's BootCD 15.2 y Knoppix 8.6.

5.2.1. Validación operativa de Hiren's BootCD 15.2

Dado que Hiren's BootCD 15.2 fue diseñado con anterioridad al estándar UEFI, su ejecución se validó en la plataforma BIOS heredada y en QEMU configurado con SeaBIOS:

1. **Arranque de Mini Windows XP:** Al seleccionar la imagen en el menú de Ventoy, el módulo de intercepción INT 13h redirigió las lecturas hacia los sectores del archivo ISO

en el pendrive. El cargador de Windows XP cargó el archivo `boot.wim` a la memoria RAM mediante `ramdisk.sys`. En 35 segundos, el sistema presentó el escritorio gráfico de MiniXP con acceso pleno a las utilerías de diagnóstico, reconociendo el teclado y el ratón USB sin conflictos de controladores.

2. **Comprobación de Memtest86+:** Desde el menú de herramientas DOS se seleccionó Memtest86+. El software tomó el control en modo protegido, detectó la totalidad de los 4096 megabytes de memoria RAM asignados en la máquina de prueba y ejecutó los 10 patrones de verificación del algoritmo March C-. Se permitió la ejecución de un ciclo completo de comprobación (Pass 1/1), reportando cero errores de bit y una tasa sostenida de lectura de memoria de 14.2 GB/s.
3. **Comprobación de Offline NT Password & Registry Editor (`chntpw`):** Se simuló un escenario de pérdida de acceso en un disco con Windows instalado. Se inició la utilería `chntpw`, la cual montó la partición NTFS anfitriona en modo lectura/escritura, abrió la colmena de registro `config\SAM`, listó los identificadores de usuario (RID) y permitió borrar con éxito la contraseña de la cuenta del usuario local `Administrador`, guardando los cambios en los sectores del disco sin corromper la estructura de colmena de Windows.

5.2.2. Validación operativa de Knoppix 8.6

Knoppix 8.6 se evaluó tanto en modo BIOS como en modo UEFI x86_64 nativo:

- **Selección de arquitectura del núcleo:** Al arrancar, el script de Knoppix comprobó las banderas del procesador en `/proc/cpuinfo`. Al detectar soporte para 64 bits, cargó automáticamente el núcleo Linux `vmlinuz64` versión 5.3, manteniendo el espacio de usuario de 32 bits conforme a su diseño híbrido.
- **Montaje del dispositivo de bucle comprimido (`cloop`):** El kernel detectó la imagen comprimida `KNOPPIX` en el medio emulado, inicializó el controlador `/dev/cloop0` y montó el sistema de archivos de más de 10 gigabytes en sólo 12 segundos, confirmando el acceso aleatorio transparente a bloques zlib.
- **Superposición AUFS y detección de hardware:** El sistema montó el árbol de directorios superponiendo `tmpfs` sobre `cloop`. El script `knoppix-autoconfig` identificó la tarjeta gráfica virtualizada, cargó los módulos de vídeo y levantó el entorno de escritorio LXDE en resolución nativa. Se comprobó el acceso a la red cableada y la ejecución fluida de utilerías de administración y pruebas de estrés de CPU.

5.3. Comprobación de sistemas de servidor, escritorio e instalador de Windows

La última fase del protocolo consistió en verificar la operatividad de los sistemas operativos de producción y despliegue: GNU/Linux Server, Kali Linux y Microsoft Windows 10.

5.3.1. Validación de GNU/Linux Server (Ubuntu Server 22.04 LTS)

Al seleccionar la imagen de Ubuntu Server en el menú de arranque de Ventoy:

1. **Inyección en el espacio de usuario temprano:** El cargador de Ventoy transfirió el kernel `vmlinuz` y el `initramfs` a la memoria física, inyectando el script de enganche `ventoy.cpio`.

2. **Montaje en bucle de la imagen ISO:** Durante la ejecución del script `/init`, el módulo de Ventoy detectó la memoria USB, creó el dispositivo de bucle `/dev/loop0` apuntando al archivo ISO de Ubuntu Server y lo montó en el punto de montaje estándar `/cdrom`. Esto evitó que el instalador abortara con el error habitual de dispositivo de instalación no encontrado.
3. **Inicialización de servicios y consola TTY:** El proceso `systemd` (PID 1) tomó el control del sistema, ejecutó las unidades de hardware y desplegó el instalador guiado Subiquity en la consola virtual TTY1. Se verificó la correcta detección de la interfaz de red Ethernet mediante Netplan (obteniendo concesión DHCP válida) y se navegó por las pantallas de particionado manual (configuración de volúmenes LVM sobre disco virtual) sin registrar anomalías en el registro del sistema (`journalctl`).

5.3.2. Validación de Kali Linux: Modo Forense y Persistencia

Kali Linux 2024 se sometió a dos pruebas operacionales críticas:

- **Comprobación del Modo Forense (*Forensic Mode*):** Se seleccionó la opción de arranque forense en el submenú de Kali. Una vez dentro del escritorio XFCE, se abrieron terminales de consola para comprobar las directivas de seguridad:

  ```
  mount | grep -E "sd[a-z]|nvme"
  swapon -show
  ```

  El resultado fue completamente nulo: ninguno de los discos duros internos de la máquina de prueba fue montado en el árbol de directorios y ninguna partición de intercambio (*swap*) fue activada por el sistema, garantizando la preservación probatoria e inalterabilidad de los medios de almacenamiento inspeccionados.
- **Comprobación de persistencia de datos:** Se reinició la máquina y se seleccionó la opción de arranque con persistencia USB (*Live with persistence*). Ventoy procesó el archivo `ventoy.json` y enlazó el contenedor `kali_persistence.dat` como dispositivo de respaldo de OverlayFS. Para verificar la persistencia, se creó un archivo testigo en el directorio de superusuario:

  ```
  echo "Prueba de persistencia multiboot» /root/test_persistence.txt
  sync
  ```

  Tras reiniciar completamente la máquina virtual y volver a iniciar Kali Linux en modo persistente, se constató que el archivo `/root/test_persistence.txt` permanecía intacto en la memoria no volátil del contenedor ext4, confirmando la operatividad del subsistema de almacenamiento persistente.

5.3.3. Validación de Microsoft Windows 10 (32 y 64 bits)

La comprobación de Windows 10 se enfocó en corroborar que el instalador superase la barrera de detección de medios ópticos y permitiera acceder a las herramientas de despliegue y rescate:

1. **Arranque del entorno WinPE:** En modo UEFI x86_64, el gestor `bootmgr.efi` cargó la imagen `boot.wim` en memoria RAM mediante `ramdisk.sys`. La emulación de Ventoy inyectó en memoria el controlador de unidad virtual, permitiendo que el asistente

`setup.exe` localizara el archivo `install.wim` dentro de la imagen ISO sin desplegar el mensaje de error “Falta un controlador de medios que su equipo necesita”.

2. **Invocación de la consola de diagnóstico:** En la primera pantalla del instalador, se presionó la combinación de teclas `Shift + F10`, abriendo una ventana de línea de comandos de administrador (`cmd.exe`) bajo WinPE.
3. **Ejecución de herramientas de administración de disco:**

```
X:\Sources> diskpart
DISKPART> list disk
DISKPART> select disk 0
DISKPART> detail disk
```

La utilería `diskpart` identificó correctamente los discos de la máquina anfitriona y la unidad USB. Asimismo, se ejecutó `wmic os get osarchitecture`, confirmando la ejecución del núcleo NT en modo nativo de 64 bits (en la ISO x64) y en modo de 32 bits (al ejecutar la variante x86 en la máquina BIOS).

La Tabla 4 sintetiza los resultados obtenidos en el banco de pruebas, evidenciando una tasa de éxito del 100 % en todas las modalidades evaluadas.

Tabla 4: Matriz de verificación funcional y pruebas de operatividad del medio multiboot.

Entorno / Sistema	**Modo de Firmware**	**Criterio de Aceptación Técnica**	**Resultado y Estado**
Menú Ventoy	UEFI / BIOS dual	Inicialización GOP 1024×768, procesamiento de alias y temporizador de 15 segundos.	Exitoso; navegación fluida en ambas plataformas.
Hiren's BootCD 15.2	BIOS Legacy / CSM	Arranque de MiniXP en RAM; 0 errores en Memtest86+; restablecimiento de contraseña en SAM.	Exitoso; control de puertos I/O y modo protegido x86.
Knoppix 8.6	UEFI / BIOS dual	Detección de CPU de 64 bits; montaje de imagen cloop con AUFS; carga gráfica de LXDE.	Exitoso; inicio en 12 segundos sin el gestor systemd.
Linux Server (Ubuntu)	UEFI x86_64	Enlace loopback en early userspace; inicialización de systemd y despliegue Subiquity en TTY1.	Exitoso; red Netplan y particionado LVM operativos.
Kali Linux	UEFI x86_64	Aislamiento forense estricto (sin montaje de discos locales ni swap); persistencia LUKS/ext4.	Exitoso; archivo testigo preservado tras reinicio.
Windows 10 (32/64 bits)	UEFI (x64) / BIOS (x86)	Carga WinPE; localización de install.wim sin fallos de medio; acceso a diskpart vía Shift+F10.	Exitoso; soporte dual x86/x64 validado al 100 %.

6. Evaluación comparativa de interoperabilidad y rendimiento

El diseño de un medio de arranque múltiple destinado a labores de diagnóstico, soporte y administración de sistemas en entornos heterogéneos exige evaluar no solo la funcionalidad individual de cada imagen integrada, sino también su grado de adaptabilidad ante la fragmentación de estándares de firmware y la sobrecarga que impone la emulación de dispositivos de bloques en tiempo de ejecución. Esta sección presenta un análisis cuantitativo y cualitativo de la interoperabilidad del pendrive multiboot, contrastando la compatibilidad de cada entorno frente a las distintas clases de arquitectura y evaluando el rendimiento del subsistema de entrada y salida (E/S) en función del bus de comunicación y los mecanismos de asignación de memoria.

6.1. Matriz de compatibilidad entre arquitecturas y modos de firmware

La transición gradual desde las interfaces de firmware tradicionales hacia los estándares contemporáneos ha generado un ecosistema fragmentado donde conviven equipos con modos de inicialización incompatibles entre sí. Para estructurar la evaluación, la especificación UEFI define una taxonomía formal basada en clases de plataforma (Unified Extensible Firmware Interface Forum, 2024):

1. **UEFI Clase 0:** Plataformas basadas exclusivamente en arquitectura BIOS tradicional de 16 bits, donde el procesador inicia en modo real y la gestión del hardware depende estrictamente de interrupciones vectorizadas en la tabla IVT (en particular INT 10h para vídeo e INT 13h para almacenamiento de bloques).
2. **UEFI Clase 1:** Implementaciones tempranas de firmware EFI que operan únicamente emulando una BIOS convencional a través del Módulo de Soporte de Compatibilidad (CSM), sin exponer interfaces EFI a los cargadores de arranque del sistema operativo.
3. **UEFI Clase 2:** Firmware híbrido que expone las interfaces nativas de servicios de arranque (*boot services*) y servicios en tiempo de ejecución (*runtime services*), manteniendo de manera opcional una subrutina CSM que permite al usuario alternar entre arranque EFI nativo y emulación BIOS de 16 bits.
4. **UEFI Clase 3:** Arquitectura contemporánea que elimina por completo el código CSM y la compatibilidad con el modo real x86, admitiendo únicamente binarios ejecutables con formato PE/COFF firmados e invocados a través de tablas de servicios EFI de 64 bits (o de 32 bits en procesadores de baja potencia).
5. **UEFI Clase 3+:** Plataformas Clase 3 donde la directiva de arranque seguro (Secure Boot) se encuentra permanentemente activa y forzada por hardware, prohibiendo la carga de cualquier binario cuya firma criptográfica no esté validada contra los certificados de la memoria NVRAM.

Al evaluar los seis entornos integrados en el medio multiboot frente a esta clasificación, surgen divergencias arquitectónicas determinantes para la administración de sistemas:

- **Hiren's BootCD 15.2:** Presenta una restricción absoluta hacia plataformas UEFI Clase 0 y Clase 2 con CSM habilitado. Debido a que el cargador Mini Windows XP, las utilerías basadas en DOS y herramientas como Memtest86+ o MHDD se diseñaron asumiendo el control de los primeros 1024 kilobytes de memoria física mediante llamadas a la BIOS, su ejecución en equipos UEFI Clase 3 resulta inviable de forma nativa (Hiren's BootCD Team, 2012). Si se requiere diagnosticar una estación de trabajo

que carece de CSM, debe recurrirse a la virtualización intermedia o a herramientas con ejecutables EFI nativos.

- **Knoppix 8.6:** Exhibe un nivel de interoperabilidad sobresaliente al incorporar binarios de arranque duales. Su medio integra tanto el cargador Syslinux/ISOLINUX para arquitecturas de 16/32 bits bajo BIOS como binarios GRUB de 64 bits para UEFI nativo (Knopper, 2019). Esto le permite arrancar en plataformas Clase 0, 1, 2 y 3. No obstante, frente a Clase 3+ con Secure Boot activo, su inicialización se bloquea salvo que se desactive la verificación en el firmware o se matricule la clave pública mediante MokManager, debido a que Knoppix no incluye de fábrica un cargador intermedio con la firma comercial de Microsoft.
- **GNU/Linux Server (Ubuntu 22.04 LTS) y Kali Linux:** Ambos entornos implementan de origen el cargador intermedio *shim* firmado por la Autoridad Certificadora de Microsoft (Microsoft Third Party UEFI CA) (Canonical Ltd., 2024; Offensive Security, 2024). Esto garantiza su ejecución transparente en plataformas contemporáneas UEFI Clase 2, Clase 3 y Clase 3+ sin exigir modificaciones en las directivas de seguridad de la máquina anfitriona. Adicionalmente, el núcleo Linux monolítico de 64 bits incluye controladores universales para controladoras de almacenamiento AHCI, NVMe y SCSI, garantizando la detección del almacenamiento secundario en cualquier servidor moderno.
- **Microsoft Windows 10 (32 frente a 64 bits):** La variante de 64 bits ofrece soporte nativo para UEFI Clase 2 y Clase 3+ con Secure Boot habilitado por defecto mediante las firmas oficiales de Microsoft en la base de datos `db` del firmware (Microsoft Corporation, 2024b). Por su parte, la variante de 32 bits (x86) se encuentra severamente acotada: no puede arrancar en plataformas UEFI Clase 3 de 64 bits debido a la discordancia estructural entre el ancho de palabra del firmware (punteros de 64 bits) y el modo de direccionamiento del gestor de arranque (`bootmgr.efi` de 32 bits). En consecuencia, Windows 10 x86 queda relegado a equipos con BIOS heredada (Clase 0/2) o a placas base excepcionales con firmware UEFI nativo IA32.

La Tabla 5 resume la compatibilidad cruzada de cada entorno frente a las distintas arquitecturas de firmware y las directivas de Secure Boot.

6.2. Análisis de sobrecarga de emulación y rendimiento de E/S

La viabilidad práctica de un medio multiboot depende fundamentalmente de la sobrecarga computacional que impone la capa de abstracción sobre los recursos del sistema anfitrión, particularmente en la memoria principal (RAM) y en el canal de transferencia del bus serie universal.

6.2.1. Sobrecarga de memoria: Mapeo de descriptores frente a disco RAM en bloque

En las arquitecturas tradicionales de arranque múltiple (como las implementadas por herramientas basadas en `memdisk` de Syslinux o la directiva `map -mem` de GRUB4DOS), el arranque desde una imagen ISO requiere transferir la totalidad del archivo desde el almacenamiento secundario hacia un búfer contiguo reservado en la memoria RAM antes de iniciar la ejecución del kernel (Silberschatz et al., 2006).

Si bien este enfoque garantiza un aislamiento absoluto de los sectores y acelera las operaciones de lectura posteriores (al servirse directamente desde la memoria dinámica a

velocidades de bus de memoria de varios gigabytes por segundo), introduce dos desventajas críticas que limitan su aplicación en entornos de ingeniería:

1. **Requisito desproporcionado de memoria física:** Para arrancar una imagen ISO de instalación de Windows 10 de 5.7 GB, el equipo anfitrión debe disponer de al menos 8 GB o 12 GB de memoria RAM física libre; de lo contrario, la reserva de memoria falla por agotamiento de espacio direccionable.
2. **Retardo inicial severo:** La copia secuencial previa de archivos de varios gigabytes a través de un bus USB introduce demoras de inicialización de varios minutos antes de que el usuario visualice la primera pantalla del sistema.

En contraste con este modelo, la tecnología empleada en el presente desarrollo no clona el medio óptico en memoria. En su lugar, el cargador de arranque lee la tabla de asignación de archivos del sistema exFAT en la Partición 1, localiza los bloques físicos que componen el archivo ISO y genera en memoria una tabla de descriptores de sector con los intervalos LBA contiguos correspondientes (Ventoy Project, 2024a).

La sobrecarga de memoria asociada a esta estructura es prácticamente despreciable: para una imagen de 6 GB fragmentada en 20 extensiones continuas, la tabla de descriptores consume menos de 2 kilobytes de RAM. Incluso sumando los controladores de intercepción en tiempo de ejecución (el módulo de enganche INT 13h en el área EBDA para BIOS o los protocolos de paso directo en el espacio EFI para UEFI), el consumo total en memoria RAM se mantiene estrictamente por debajo de los 10 MB. Esto permite inicializar sistemas operativos complejos e imágenes pesadas en estaciones de trabajo y servidores dotados de recursos mínimos de memoria (por ejemplo, equipos de diagnóstico con únicamente 2 o 4 gigabytes de RAM física).

6.2.2. Rendimiento de transferencia y latencia en buses USB 2.0 y USB 3.0

El factor determinante en la velocidad de inicialización y respuesta de los sistemas operativos en modo de ejecución directa radica en el ancho de banda del canal de comunicación entre el controlador del anfitrión y la memoria flash del pendrive (Stallings, 2018, §11.2, p. 500):

- **Bus USB 2.0 (HighSpeed):** Opera con una tasa de señalización teórica de 480 Mbps gestionada por controladores EHCI (*Enhanced Host Controller Interface*). Debido a la sobrecarga del protocolo por sondeo unidireccional y las limitaciones de sincronización de paquetes por tramas de 1 milisegundo, la tasa sostenida de lectura secuencial se sitúa entre 30 MB/s y 38 MB/s. En este escenario, el tiempo de transferencia para levantar la imagen WinPE de Windows 10 (450 MB) promedia los 48 segundos, mientras que la inicialización del árbol de directorios de Knoppix alcanza los 35 segundos.
- **Bus USB 3.0 / 3.1 Gen 1 (SuperSpeed):** Incrementa la velocidad de señalización a 5 Gbps mediante la arquitectura xHCI (*Extensible Host Controller Interface*). Esta interfaz implementa canales dedicados de transmisión y recepción full-duplex y permite la operación del protocolo UASP (*USB Attached SCSI Protocol*), el cual habilita el encolamiento de comandos por hardware (NCQ) y elimina la sobrecarga del protocolo de sondeo masivo. La tasa sostenida de lectura secuencial sobre el pendrive utilizado alcanza entre 140 MB/s y 210 MB/s. En consecuencia, el tiempo de carga de WinPE se reduce a solo 13 segundos (una ganancia del 73 % en velocidad de respuesta), y el montaje del sistema comprimido de Knoppix desciende a 11 segundos.

6.2.3. Preservación de vida útil de la memoria flash frente a métodos tradicionales

Desde la perspectiva del ciclo de vida del hardware de almacenamiento, la solución multiboot aporta un beneficio sustancial en comparación con utilerías clásicas como Rufus en modo de escritura DD (Batard, 2024).

En el esquema tradicional de un único sistema por medio, cada vez que el administrador necesita sustituir un sistema operativo por otro, la herramienta debe formatear íntegramente la unidad USB o sobreescribir masivamente los bloques de datos y las tablas de particiones sector por sector. Las memorias flash NAND sufren un desgaste físico irreversible en sus celdas de almacenamiento con cada ciclo de borrado y reprogramación de bloques (*P/E cycles*, Program/Erase), cuyo límite típico en chips de grado comercial oscila entre 1 000 y 3 000 ciclos por celda (Silberschatz et al., 2006, §12.1, p. 390).

Al operar sobre un sistema de archivos estructurado exFAT donde las imágenes ISO coexisten como archivos ordinarios, el medio multiboot desarrollado elimina por completo la necesidad de reformateos sucesivos. La adición, actualización o eliminación de un sistema operativo se reduce a una operación atómica a nivel de sistema de archivos, concentrando las escrituras exclusivamente en los bloques requeridos y preservando la vida útil del almacenamiento flash para un uso prolongado en entornos de campo.

Tabla 5: Matriz comparativa de interoperabilidad de firmware, seguridad y métricas de rendimiento.

Entorno Integrado	Soporte de Firmware	Compatibilidad Secure Boot	Sobrecarga de RAM	Tiempo de Arranque (USB 3.0)
Hiren's BootCD 15.2	BIOS Legacy / UEFI CSM (Clase 0/2)	Incompatible; requiere desactivar verificación.	< 5 MB (mapeo) + 150 MB (MiniXP RAM).	28 segundos (MiniXP en memoria física).
Knoppix 8.6	BIOS y UEFI dual (Clase 0, 1, 2, 3)	Requiere clave MOK o desactivar Secure Boot.	< 8 MB (mapeo) + descompresión al vuelo.	12 segundos (LXDE nativo con cloop).
Linux Server (Ubuntu 22.04)	UEFI nativo (Clase 2, 3 y 3+)	Compatible de fábrica (Shim de Microsoft).	< 6 MB (mapeo) + búfer initramfs.	16 segundos (consola TTY Subiquity).
Kali Linux 2024	UEFI nativo (Clase 2, 3 y 3+)	Compatible mediante MOK / Shim integrado.	< 8 MB (mapeo) + OverlayFS en RAM/ext4.	18 segundos (escritorio XFCE Live).
Windows 10 x64	UEFI nativo (Clase 2, 3 y 3+)	Compatible de fábrica (certificados oficiales).	< 10 MB (mapeo) + 500 MB (WinPE RAM).	14 segundos (asistente de instalación).
Windows 10 x86	BIOS Legacy y UEFI IA32 (Clase 0/2)	Restringido a entornos de 32 bits compatibles.	< 8 MB (mapeo) + 400 MB (WinPE RAM).	17 segundos (asistente de instalación).

7. Conclusiones personales

La arquitectura y desarrollo de un medio de arranque múltiple permite comprender, desde una perspectiva integradora, los fundamentos que articulan la interacción entre el firmware, el cargador de arranque y el núcleo del sistema operativo. La superación del paradigma monoboot convencional, que históricamente forzaba a formatear y reconstruir la tabla de particiones del medio de almacenamiento ante cada cambio de imagen de instalación, representa una evolución sustancial en la abstracción del almacenamiento masivo. Al disociar la estructura de bloques físicos del medio flash de la geometría lógica de las imágenes de disco mediante un sistema de archivos estructurado como exFAT, se evidencia la potencia de los principios de diseño de los sistemas operativos modernos: interponer capas de indirección que permitan desacoplar las exigencias del software de las limitaciones directas impuestas por el hardware subyacente.

El análisis de la fragmentación de plataformas puso en relieve la marcada brecha arquitectónica que separa a los sistemas basados en BIOS heredada de las implementaciones contemporáneas de UEFI. Durante más de cuatro décadas, el ecosistema informático dependió de convenciones heredadas del modo real de 16 bits de la arquitectura Intel x86, condicionando la fase de inicialización a estructuras estáticas como la tabla de vectores de interrupción y el área de datos de la BIOS extendida. La transición hacia el estándar UEFI y su paulatina consolidación en plataformas de Clase 3 (las cuales suprimen definitivamente el módulo de compatibilidad CSM) representa una ruptura determinante con dicha herencia. La imposibilidad de ejecutar entornos clásicos como Hiren's BootCD 15.2 en máquinas Clase 3 sin la intermediación de un hipervisor demuestra que el mantenimiento de sistemas heredados debe gestionarse con una clara conciencia de las fronteras del hardware, demandando la migración progresiva hacia herramientas de diagnóstico dotadas de binarios EFI nativos de 64 bits.

La técnica de intercepción en tiempo de ejecución implementada por Ventoy sobresale como una solución de ingeniería de notable elegancia frente a los métodos de clonación masiva en disco de memoria. Las aproximaciones tradicionales basadas en utilerías de arranque forzaban la copia íntegra del archivo ISO a la memoria principal antes de transferir el control al kernel, lo cual no solo demoraba el inicio de los sistemas durante varios minutos, sino que además imponía una barrera de hardware infranqueable al requerir que el equipo dispusiera de tanta memoria RAM libre como el tamaño del medio emulado. En contraposición, el mapeo dinámico de intervalos LBA contiguos y la intercepción de llamadas de lectura (a través de la interrupción 13h en BIOS o de los protocolos de bloques en UEFI) reducen la sobrecarga de memoria a unos pocos kilobytes para tablas de descriptores y menos de diez megabytes para el módulo de enganche. Este mecanismo democratiza el soporte técnico al hacer viable el arranque de instaladores y sistemas complejos de varios gigabytes en equipos con dotaciones mínimas de memoria RAM.

Uno de los problemas arquitectónicos más desafiantes abordados en este desarrollo radica en la transición de control desde el firmware hacia el espacio de usuario temprano del núcleo del sistema operativo. Debido a que los núcleos modernos de Linux y los entornos WinPE de Windows asumen el control de los controladores de almacenamiento y prescinden de los servicios de arranque del firmware inmediatamente después de inicializar la gestión de memoria protegida, la intercepción de bloques en memoria pierde vigencia si no se interviene el proceso de arranque en el espacio de usuario. La inyección de scripts especializados en la imagen del initramfs (como el archivo de enganche de Ventoy) y la incorporación de controladores virtuales en el entorno WinPE resuelven con absoluta trans-

parencia el clásico fallo de desconexión del medio de instalación. El enlace dinámico de dispositivos de bucle hacia puntos de montaje normalizados garantiza que el instalador acceda a sus paquetes base sin alterar los binarios originales de las distribuciones.

La evaluación de los sistemas operativos en modo de ejecución directa puso de manifiesto la versatilidad de los sistemas de archivos comprimidos y en capas. Distribuciones pioneras en este enfoque, como Knoppix, demostraron la viabilidad de descompresión al vuelo de bloques mediante el controlador cloop y la superposición de directorios en memoria con AUFS, logrando empaquetar estaciones de trabajo completas en espacios de almacenamiento reducidos sin penalizar el rendimiento interactivo. Asimismo, los esquemas contemporáneos basados en OverlayFS y SquashFS, presentes en GNU/Linux Server y Kali Linux, perfeccionan este paradigma al integrarse de forma nativa en la línea principal del núcleo Linux. Esta arquitectura permite que sistemas operativos de solo lectura operen con pleno dinamismo, sirviéndose de la memoria RAM como capa de escritura efímera y ofreciendo entornos inmutables y resilientes ante fallos de software.

La contrastación de modalidades en Kali Linux reveló dos filosofías operativas de profunda relevancia para la administración y la seguridad informática. Por un lado, el modo forense ilustra el rigor probatorio exigido en labores de peritaje y respuesta a incidentes, donde la directiva estricta de no automontar ningún medio de almacenamiento interno ni activar particiones de intercambio asegura la inalterabilidad de la evidencia digital. Por otro lado, la integración de la persistencia mediante contenedores ext4 cifra un puente funcional entre la portabilidad y la personalización, permitiendo a los especialistas en ciberseguridad preservar configuraciones, herramientas y registros de auditoría entre reinicios sin vulnerar la integridad de la imagen original. Esta dualidad evidencia que el comportamiento de un mismo núcleo monolítico puede orientarse hacia objetivos diametralmente opuestos mediante la simple manipulación de los parámetros de línea de comandos pasados por el cargador de arranque.

En el ámbito del sistema operativo Windows 10, la coexistencia de las variantes de 32 y 64 bits expuso las divergencias medulares que determinan la evolución del núcleo Windows NT. La variante de 64 bits no solo amplía el espacio de direccionamiento virtual a niveles capaces de solventar la saturación histórica de los grupos de memoria no paginada, sino que incorpora mecanismos mandatorios de seguridad activa como la firma obligatoria de controladores en modo kernel y la protección contra modificaciones del núcleo mediante PatchGuard. La inviabilidad técnica de arrancar la edición de 32 bits en placas base con firmware UEFI de 64 bits subraya la rigidez de las convenciones de llamada y los tamaños de puntero en los servicios de arranque. Esto constata que la compatibilidad en sistemas de 64 bits se garantiza a nivel de espacio de usuario mediante subsistemas de emulación como WoW64, mientras que en el estrato de prearranque y del núcleo la coincidencia entre el ancho de palabra del firmware y el del sistema operativo es mandatoria e ineludible.

El análisis de la infraestructura de Secure Boot permitió desmitificar la concepción habitual de que las medidas criptográficas de seguridad en el prearranque constituyen un obstáculo insalvable para el software libre y las herramientas de administración independiente. Fundamentado en una jerarquía de certificados digitales X.509 almacenados en memoria NVRAM, el arranque seguro actúa como una barrera efectiva contra amenazas avanzadas como rootkits y bootkits que intentan corromper el cargador antes de la inicialización de las directivas del sistema operativo. La articulación del cargador intermedio shim, firmado por entidades de certificación reconocidas, combinada con la utilidad MokManager, restituye al administrador del equipo el control legítimo sobre su hardware. Esta solución permite matricular certificados personalizados y ejecutar entornos confiables sin necesidad de degradar

la postura de seguridad de la máquina desactivando la verificación en el firmware.

Desde el punto de vista del rendimiento del subsistema de entrada y salida, la disparidad observada entre los estándares USB 2.0 y USB 3.0 resalta la trascendencia de los protocolos de comunicación y la arquitectura de los controladores anfitriones. La transición desde el protocolo de sondeo unidireccional de los controladores EHCI hacia la arquitectura full-duplex de los controladores xHCI y el protocolo UASP transforma sustancialmente la experiencia operativa de los sistemas en modo en vivo. La reducción de más del setenta por ciento en el tiempo de carga del entorno WinPE y de los paquetes base de Linux demuestra que el canal de comunicación y la tasa de transferencia sostenida representan el verdadero cuello de botella en los medios de arranque externos, superando en impacto a la potencia de cómputo del procesador anfitrión.

Asimismo, la preservación del soporte físico constituye un beneficio directo derivado de la arquitectura basada en sistemas de archivos. El desgaste irreversible de las celdas de almacenamiento flash NAND debido a los ciclos de borrado y reprogramación de bloques representa una limitación física ineludible en dispositivos de estado sólido. Al evitar los formateos masivos continuos y las escrituras directas sector por sector que caracterizan a las herramientas tradicionales de un solo sistema por medio, el enfoque desarrollado prolonga sustancialmente la vida útil de la memoria USB. La posibilidad de añadir, eliminar o actualizar imágenes ISO como simples archivos ordinarios minimiza las operaciones de escritura innecesarias y reduce el riesgo de fallas prematuras del dispositivo en tareas de campo intensivas.

La integración armónica de entornos tan dispares en un único medio portable —desde utilerías que manipulan directamente registros y puertos de entrada y salida hasta plataformas de servidores contemporáneas orquestadas por systemd y redes Netplan— consolida una herramienta indispensable para el ejercicio profesional de la ingeniería en computación. La capacidad de diagnosticar módulos de memoria RAM, auditar estructuras complejas como la base de datos SAM de Windows, evaluar vulnerabilidades en infraestructuras de red o desplegar sistemas de servidor en entornos de producción exige una comprensión integral de las capas que conforman la computación moderna.

En conclusión, el desarrollo de este proyecto demuestra que un medio de arranque múltiple no debe concebirse como un simple repositorio de archivos ejecutables, sino como una solución de ingeniería de sistemas que sintetiza de manera práctica los conceptos medulares de los sistemas operativos. Comprender el trayecto que recorre una instrucción desde la inicialización del vector de reinicio de la CPU hasta la transferencia de control al proceso de inicio en el espacio de usuario proporciona los criterios teóricos y operativos necesarios para diagnosticar fallas críticas, diseñar arquitecturas resilientes y optimizar el rendimiento en infraestructuras tecnológicas complejas y heterogéneas.

Referencias

Batard, P. (2024). *Rufus: The reliable usb formatting utility*. Consultado el 1 de noviembre de 2026, desde https://github.com/pbatard/rufus/wiki/FAQ

Canonical Ltd. (2024). *Ubuntu server guide: System architecture, administration and storage*. Consultado el 1 de noviembre de 2026, desde https://ubuntu.com/server/docs

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson/Prentice Hall.

Free Software Foundation. (2024a). *Gnu grub manual 2.12*. Consultado el 1 de noviembre de 2026, desde https://www.gnu.org/software/grub/manual/grub/grub.html

Free Software Foundation. (2024b). *Multiboot specification version 0.6.96*. Consultado el 1 de noviembre de 2026, desde https://www.gnu.org/software/grub/manual/multiboot/multiboot.html

Hiren's BootCD Team. (2012). *Hiren's bootcd 15.2 release notes and utility overview*. Consultado el 1 de noviembre de 2026, desde https://www.hirensbootcd.org/hbcd-v152/

Knopper, K. (2019). *Knoppix 8.6.0 linux live system release notes*. Consultado el 1 de noviembre de 2026, desde https://www.knopper.net/knoppix/knoppix860-en.html

Knopper, K. (2024). *Compressed loopback device (cloop)*. Consultado el 1 de noviembre de 2026, desde https://www.knopper.net/knoppix/knoppix-cloop-en.html

Landley, R., & The Linux Kernel Organization. (2024). *Ramfs, rootfs and initramfs*. Consultado el 1 de noviembre de 2026, desde https://docs.kernel.org/filesystems/ramfs-rootfs-initramfs.html

Microsoft Corporation. (2024a). *Overview of boot options in windows*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-hardware/drivers/bringup/boot-options-in-a-multiboot-environment

Microsoft Corporation. (2024b). *Windows 10 architecture and deployment overview*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows/deployment/windows-10-deployment-scenarios

Microsoft Corporation. (2024c). *Windows pe (winpe) technical reference*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-hardware/manufacture/desktop/winpe-intro

Offensive Security. (2024). *Kali linux documentation: Live boot, persistence and system architecture*. Consultado el 1 de noviembre de 2026, desde https://www.kali.org/docs/

Pen Drive Linux. (2024). *Yumi: Multiboot usb creator*. Consultado el 1 de noviembre de 2026, desde https://pendrivelinux.com/yumi-multiboot-usb-creator/

QEMU Project. (2024). *Qemu system emulation user documentation: Kvm, storage and device passthrough*. Consultado el 1 de noviembre de 2026, desde https://www.qemu.org/docs/master/system/index.html

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

Unified Extensible Firmware Interface Forum. (2024). *Unified Extensible Firmware Interface (UEFI) Specification* (inf. téc. N.º Version 2.10). UEFI Forum. Consultado el 1 de noviembre de 2026, desde https://uefi.org/specifications

Ventoy Project. (2024a). *Ventoy documentation and architecture description*. Consultado el 1 de noviembre de 2026, desde https://www.ventoy.net/en/doc_start.html

Ventoy Project. (2024b). *Ventoy plugin documentation: Persistence, control and auto-installation*. Consultado el 1 de noviembre de 2026, desde https://www.ventoy.net/en/plugin_entry.html